Wazuh FIM - File Integrity Monitoring
The Advanced Defender's Handbook

Wazuh 4.14.8 (4.14.x series) | syscheck: scheduled, realtime and whodata | Windows and Linux

Author: Kislley Rodrigues | Wazuh Ambassador Program | October 2026
GitHub: https://github.com/mym0us3r/wazuh-fim-adh

Classification	**Technical Handbook on Detection and Response with FIM in Wazuh**
Reference version	Wazuh Server and agents 4.14.8 (4.14.x series)
Platforms	Windows 11 Pro (labdesk) | Ubuntu 24.04.5 (labnix) | Kali Linux (pentlab)
Contents	11 custom rules | 15 technical findings | MITRE ATT&CK | Compliance
Evidence	Real screenshots from Wazuh Discover and the FIM Alert Investigation panel (AI-assisted)
Authorship	Kislley Rodrigues - Wazuh Ambassador Program
Date	October 2026

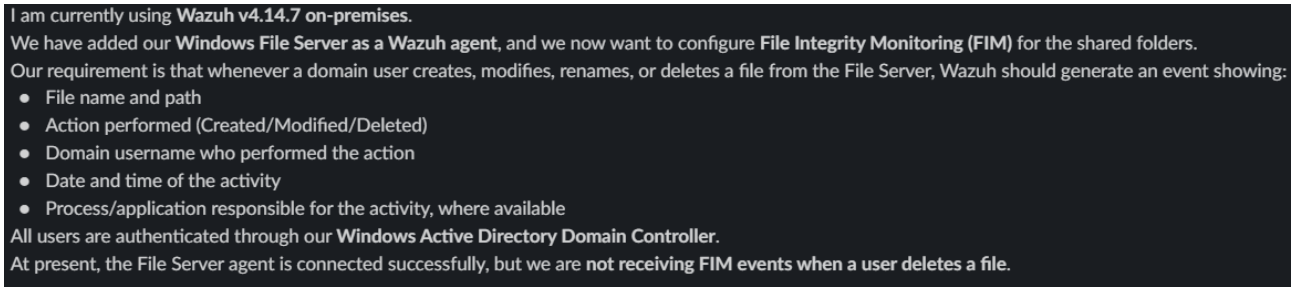
1. Executive Summary

The **File Integrity Monitoring** (FIM) module in Wazuh detects what changed, when it changed and, with whodata, who changed it. In production, however, most coverage incidents stem from three causes that are often overlooked: the operating system audit subsystem underneath FIM, the way wazuh-analysisd works when correlating events, and certain configuration conditions that reduce coverage without logging any error.

This handbook consolidates in a single document the knowledge validated in a dedicated lab about FIM in Wazuh 4.14.x: the architecture and the three monitoring modes, the implementation on Windows and Linux, a set of six correlation rules for behavioral anomaly detection in file systems, the **FIM Alert Investigation** panel with LLM-assisted analysis, and the 15 validated technical findings.

The same questions about File Integrity Monitoring (FIM) come up repeatedly in the Wazuh community, on Slack, in forums, on LinkedIn and on other platforms, especially among users deploying FIM in production for the first time. The pattern is usually very similar: creation and modification events are received normally, but deletion events never arrive, or the behavior is exactly the opposite.

Given this scenario, it is common to assume that the solution is a single missing line in ossec.conf. In some cases, that assumption is correct. In many others, however, the cause lies in a layer below the FIM configuration itself, either in the operating system audit subsystem or in a Group Policy that is still applied to the server OU, even though it has been forgotten or goes unnoticed during the investigation.


I am currently using **Wazuh v4.14.7 on-premises**.
We have added our **Windows File Server as a Wazuh agent**, and we now want to configure **File Integrity Monitoring (FIM)** for the shared folders.
Our requirement is that whenever a domain user creates, modifies, renames, or deletes a file from the File Server, Wazuh should generate an event showing:

- File name and path
- Action performed (Created/Modified/Deleted)
- Domain username who performed the action
- Date and time of the activity
- Process/application responsible for the activity, where available

All users are authenticated through our **Windows Active Directory Domain Controller**.
At present, the File Server agent is connected successfully, but we are **not receiving FIM events when a user deletes a file**.

Figure 1 - Recurring question in the Wazuh community: agent on a Windows File Server (Wazuh 4.14.7) with no FIM events

This scenario was also one of the main motivations for writing this handbook. The expectation is that it serves as a clear, objective quick-reference guide, helping to answer the most common questions from those who do not yet have advanced knowledge of Wazuh, especially in the context of FIM (File Integrity Monitoring). This material gathers and organizes the main aspects of the FIM architecture, detailing where each configuration component is located, both on the Wazuh server and on the Windows and Linux endpoints. It also covers how Linux Audit rules are actually built and persisted, as well as the failure modes that explain most of the reports of deletion events that never reach Wazuh, as observed in community discussions. All command and path examples presented in this material use Wazuh Server 4.14.8 as the reference. The technical content was validated in a dedicated lab made up of one Windows agent and one Linux agent, both monitored by the same Wazuh Server.

> **WARNING:** FIM can end up without coverage with no error logged. Two cases were validated in the lab: on Windows, the whodata SACL is not restored after a manager restart (Finding AO-7); on Linux, a path with `/*` in `<directories>` registers watches on individual files and leaves new files without realtime coverage (Finding AO-8). Evidence and mitigations are in chapter 10.

1.1 Key Results

Result	Detail	Status
Custom correlation rules	100300, 100301, 100302, 100303, 100304 and the proxy rule 100310 in `fim_anomaly_rules.xml`	**VALIDATED**
Counter isolation in analysisd	Proxy rule 100310 separates the accumulators of 100300 and 100301 (groups `fim_creation` and `fim_new_entry`)	**VALIDATED**
Cross-platform validation	Windows (LABDESK) and Linux (pentlab and labnix) for the creation, modification, deletion and escalation rules	**VALIDATED**
AI-assisted investigation	FIM Alert Investigation with Claude Sonnet 4.6: FIM-to-verdict latency of 16 s measured on labnix	**VALIDATED**
Windows post-exploitation simulation	44 FIM alerts, rule 100302 triggered, `risk_score` 87 (CRITICAL) verdict with 11 MITRE techniques	**VALIDATED**
Rule 100301 on Windows	`same_field` on syscheck fields not accessible to the correlation engine in the tests of 2026-09-16; limitation documented, with creation coverage via 100300 on Windows	**PARTIAL**

Table 1 - Summary of the results validated in the lab

CALIBRATION NOTICE: The `frequency="3"`, `timeframe="600"` and `ignore="30"` parameters of the correlation rules were calibrated for the lab. In production, tune them against a baseline of normal activity per agent group before deployment.

1.2 Handbook Structure

Chapter	Content
2. FIM Fundamentals	Architecture, where the configuration lives, the three modes, parameters and the network path restriction
3. FIM on Windows	SACL, audit subcategories, `auditpol`, GPO, rename and Registry monitoring
4. FIM on Linux	auditd, persistent rules, audit-keys CDB, eBPF provider, inotify and diagnostic checklist
5. Anomaly Detection Rules	Six correlation rules, the proxy pattern of rule 100310 and the evidence per platform
6. FIM Alert Investigation	Architecture, threat intelligence enrichment, LLM verdict and validations
7. Complementary Rules	Native rules that determine FIM resilience: agent lifecycle, inotify queue and Registry, plus the Active Response reference
8. Adversary Perspective	Monitoring evasion, timestomping and the ransomware event pattern
9. Compliance and MITRE ATT&CK	PCI DSS 4.0, NIST 800-53, HIPAA, GDPR, TSC and technique mapping
10. Technical Findings	The 15 validated findings, with evidence, impact and mitigation
11. Reference Configurations	What is new in 4.14.8, hardened ossec.conf, server state and conclusion
12. Official References	Verified Wazuh documentation and project references

Table 2 - Chapter map

1.3 Reference Environment

The lab runs a Wazuh server and agents on different platforms, which makes it possible to validate each scenario across heterogeneous ecosystems. No network address of the server or of production environments appears in this document; address fields use placeholders.

Component	Platform	Role
WZ-SRV	Ubuntu 24.04 LTS, 3.9 GB of RAM, Wazuh 4.14.8	Wazuh server: manager, indexer, dashboard and WPTV (Flask)
LABDESK	Windows 11 Pro (build 26200), agent 4.14.8	Windows agent: FIM on file system and Registry
labnix	Ubuntu 24.04.5, kernel 6.17.0, agent 4.14.8	Current Linux agent: realtime, whodata and auditd
pentlab	Kali Linux (kernel 6.x), agent 4.14.7	Linux agent from the initial validations
Intelligence	Claude Sonnet 4.6 (Anthropic), VirusTotal, AlienVault OTX, AbuseIPDB	Analysis and enrichment for FIM Alert Investigation

Table 3 - Lab components

Two traceability notes. First, the 4.14.7 and 4.14.8 evidence coexist: part of the correlation scenarios was run on 4.14.7, and the lab's current pipeline runs on 4.14.8. Second, the lab capacity is that of a server with 3.9 GB of RAM; the volume and latency figures reported here reflect that profile and are not a substitute for production sizing.

2. Wazuh FIM Fundamentals

FIM in Wazuh is implemented by the syscheck component, which runs as the `wazuh-syscheckd` daemon on each agent. This daemon maintains a local baseline of the monitored files and directories, capturing checksums (MD5, SHA-1, SHA-256), size, permissions, owner, group and modification time, and compares the current file system state against that baseline whenever a change is detected or a scan runs.

2.1 Anatomy of Detection

Detection is a comparison between two states: the cryptographic baseline stored locally and the reality observed on disk. Any checksum or attribute divergence generates an event that the server decodes and correlates.

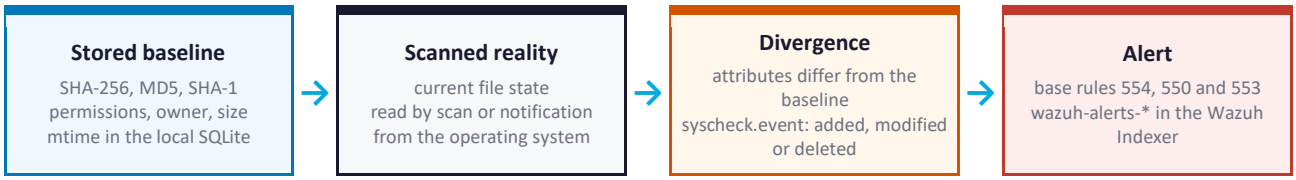


Figure 2 - Anatomy of detection: FIM compares the live state against the baseline and raises an alert when it finds a divergence

The module keeps two databases synchronized. On the endpoint, a local SQLite database stores the baseline in `queue/fim/db`, inside the agent installation directory. On the server, the `wazuh-db` daemon keeps one database per agent, used to serve FIM-related API queries and to keep the file inventory shown in the dashboard up to date. Synchronization sends only what changed: checksums and attributes that diverge from the last known state.

2.2 End-to-End Architecture

The flow, as described in the official Wazuh documentation, starts at the monitored file system, passes through the agent's local database (FIM DB), continues to the server as a FIM event message and a synchronization message, is processed by the decoder and the rule match on the manager, and ends in the agents database and the alerts file. In the lab, the Wazuh Indexer and the investigation panel are added to this flow.

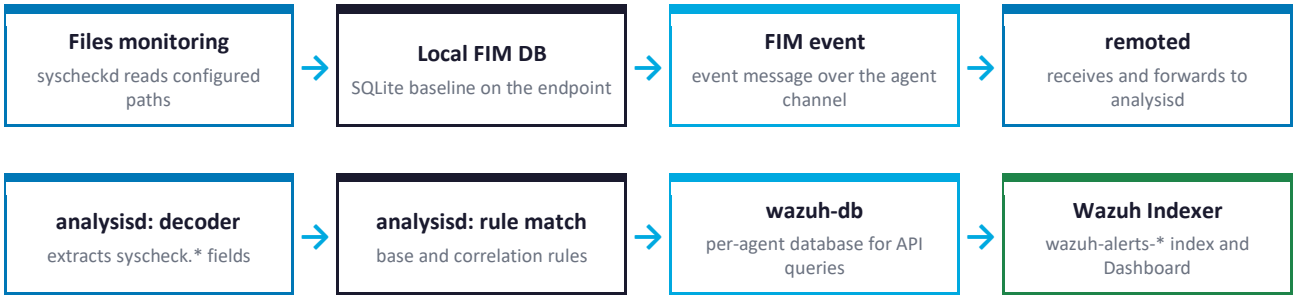


Figure 3 - End-to-end FIM flow: endpoint (top row) and server (bottom row). Based on documentation.wazuh.com, File Integrity Monitoring capability - how it works

Breaking it down: `syscheckd` runs the initial baseline scan at startup and populates the local SQLite database. From then on, depending on the mode configured for each directory, it reacts to changes through operating system notification or through periodic comparison against the baseline. When a divergence is found, the agent generates an event and sends it to the server. `wazuh-analysisd` decodes it and applies the rules; if there is a

match, the event becomes an alert, which is sent to the Wazuh Indexer for storage and to the Wazuh Dashboard for visualization.

> **Behavior on restart:** Whenever the agent service restarts, `syscheckd` rebuilds its local database and runs a new full scan, synchronizing the result with the server. Directories with realtime or whodata synchronize immediately; the others wait for the next scheduled scan. Changes that occurred while the service was stopped are not reported retroactively.

2.3 Where the Configuration Lives

Most "I configured it and nothing changed" reports come from editing the wrong file, or a file that centralized group configuration silently overwrites at the agent's next check-in. The map below avoids that mistake.

Location	Path	Note
Manager (local)	`/var/ossec/etc/ossec.conf`	Affects only the manager host
Agent group	`/var/ossec/etc/shared/<group_name>/agent.conf`	Centralized configuration, preferable to per-agent editing
Custom rules	`/var/ossec/etc/rules/`	Silence noise and correlation logic
Default ruleset	`/var/ossec/ruleset/rules/`	Includes 0015-ossec_rules.xml and 0365-auditd_rules.xml
audit-keys CDB	`/var/ossec/etc/lists/audit-keys`	Maps Audit keys to event categories
Windows agent	`C:\Program Files (x86)\ossec-agent\ossec.conf`	Restart: `Restart-Service -Name wazuh`
Linux agent	`/var/ossec/etc/ossec.conf`	Restart: `systemctl restart wazuh-agent`
macOS agent	`/Library/Ossec/etc/ossec.conf`	Restart: `/Library/Ossec/bin/wazuh-control restart`

Table 4 - Location of the FIM configuration files by component

On all three agent platforms, the configuration is edited inside the `<syscheck>` block and requires a service restart to apply changes. `syscheckd` only recognizes new directories, and only auto-configures the SACL or audit rules for them, after a restart, not on a hot reload.

2.4 The Three Monitoring Modes

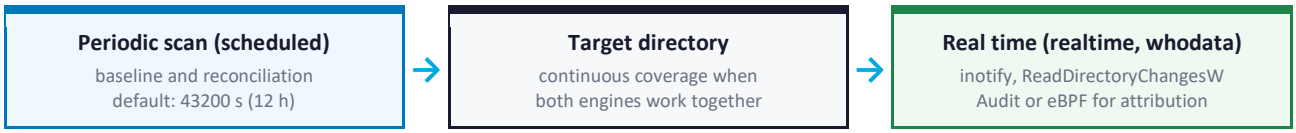


Figure 4 - Dual monitoring engine: periodic scan for baseline and real time for immediate capture, in parallel

Mode	Mechanism	User attribution	Overhead	When to use
scheduled	Full scan every `frequency` (default 43200 s) or at `scan_time` and `scan_day`	No	Lowest	Static, low-activity paths

Mode	Mechanism	User attribution	Overhead	When to use
realtime	`inotify` (Linux) and `ReadDirectoryChangesW` (Windows); directories only	No	Medium	Immediate detection without a "who" requirement
whodata	Audit subsystem: SACL on Windows; auditd or eBPF on Linux	Yes (user and process)	Highest	Compliance and incident response

Table 5 - Comparison of the syscheck monitoring modes

Scheduled scan

This is the default mode and the one with the lowest overhead. A file that is created, modified and deleted entirely between two scans produces no event, because the two compared states never captured the transient change. It is the mode many environments end up in without noticing, when a directory is added without the `realtime` or `whodata` attributes.

Real time

It uses the operating system's native notification API to detect creation, modification and deletion the instant they happen. It is enabled per directory with `realtime="yes"` and does not work on individual files. It reports what changed and when, but not who or which process, because the notification API does not carry that information.

Who-data

It adds user and process attribution on top of real-time detection, using the operating system audit subsystem. It is required whenever a requirement includes "**who made the change**", which covers most compliance-driven deployments and incident response cases. If whodata fails to initialize, the module automatically falls back to realtime, silently losing attribution while still detecting the change.

```
<syscheck>
  <directories check_all="yes" whodata="yes"
report_changes="yes">/path/to/monitored</directories>
</syscheck>
```

2.5 Configuration Parameters

Parameter	Function	Operational note
`check_all`	Checks all attributes: size, permissions, owner, group, checksums and mtime	Recommended baseline for any directory monitored seriously
`whodata`	User and process attribution	Requires OS auditing to be available (chapters 3 and 4)
`realtime`	Immediate detection through the OS API, without attribution	Redundant alongside `whodata`, which already implies real time
`report_changes`	Text content diff, keeping a copy of the file	Useful for configuration files and scripts; for binaries it consumes space in `queue/diff` with no value
`restrict`	Regular expression filter on the file name	Main tool for reducing noise in large directories
`recursion_level`	Subdirectory depth in the scan	Default 256, maximum 320; 0 monitors only the given directory
`frequency`	Scheduled scan interval, in seconds (default 43200)	Does not affect detection on realtime or whodata directories

Parameter	Function	Operational note
`scan_time`, `scan_day`	Pin the full scan to a specific time and day	Useful for maintenance windows

Table 6 - Parameters of the directories block and of syscheck

2.6 The Network Path Restriction

Starting with version 4.13.0, and kept in 4.14.x, FIM on Windows does not support UNC network paths, such as `\\server\share\folder`, nor mapped drives, such as a `Z:\folder` letter pointing to a remote host. Any path of this kind declared in the configuration is ignored and generates no events, in any mode. According to the Wazuh 4.13.0 release notes (#30060), the restriction fixes the NetNTLMv2 exposure caused by monitoring UNC paths and mapped drives.

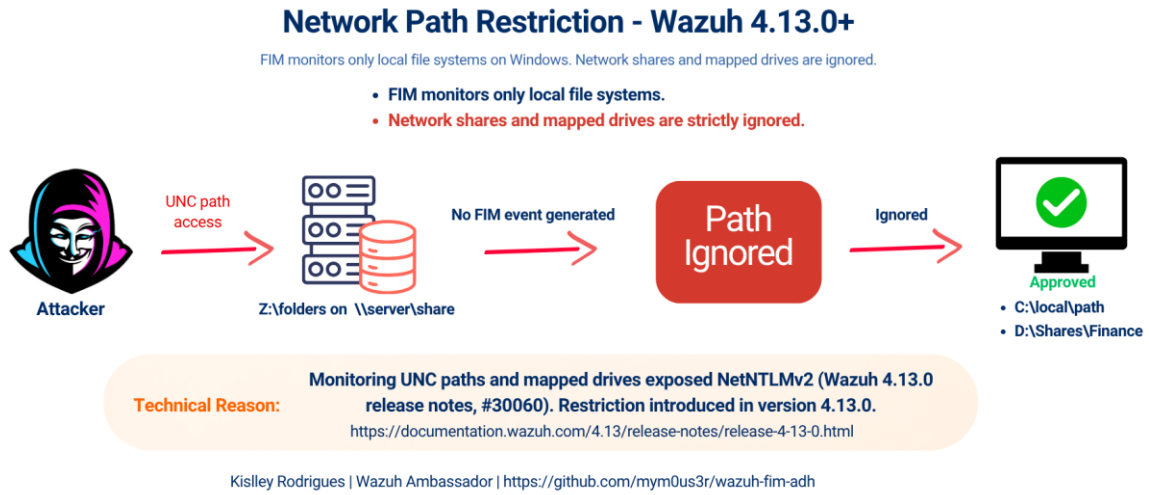


Figure 5 - Network path restriction in Wazuh 4.13.0 and later: shared folders must be monitored by their local path on the server

2.7 Delta Synchronization

The `synchronization` block inside `syscheck` does not control when FIM alerts are sent; it controls when the agent's local database is reconciled with the server's database. These are distinct mechanisms: alerts travel in real time over the event channel, while synchronization sends only deltas.



Figure 6 - Delta synchronization: unchanged data never crosses the network

```
<synchronization>
  <enabled>yes</enabled>
  <interval>5m</interval>
  <max_eps>10</max_eps>
</synchronization>
```

The 5-minute interval is the initial time between synchronizations; if one fails, the interval doubles up to max_interval (default 1h). The `max_eps` of 10 limits the rate of synchronization messages so as not to saturate `analysisd` when an agent that was offline for a long time reconnects.

INVESTIGATION IMPLICATION: If an agent was offline during an attack, reconnection does not resend the events from that period. The server calculates the delta between the state before the disconnection and the current state. A file that was created and then deleted while the agent was offline results in a zero delta and no alert. A gap in the FIM history must never be read as an absence of activity: the agent's reconnection timestamp is the first piece of data to check before any timeline analysis.

2.8 General Best Practices

- **Review frequency and use scan_time.** On paths monitored only by schedule, a lower value shortens the detection window and increases I/O proportionally; `scan_time` and `scan_day` shift the scan to a maintenance window.
- **Use restrict and ignore rigorously on noisy directories.** A directory monitored for malware persistence rarely needs to report every temporary file.
- **Reserve report_changes for where the diff has value,** avoiding unnecessary disk consumption on binary directories.
- **There is no native "renamed" event** on any platform. A rename is reported as the deletion of the old path followed by the creation of the new path; correlating them as a single action requires a custom rule.

3. FIM on Windows

This chapter covers the implementation of FIM on Windows endpoints monitored by a Wazuh Server 4.14.x: the audit policy auto-configuration performed by the agent, the most common points of failure and correct validation in a domain-joined environment.

3.1 Examples of Monitored Directories

Beyond the default set that Wazuh ships (`%WINDIR%`, `System32`, `SysNative` and the Startup folder), the examples below illustrate distinct use cases on a typical Windows endpoint.

```
<syscheck>
  <directories check_all="yes" whodata="yes"
report_changes="yes">D:\Shares\Finance</directories>
  <directories check_all="yes" whodata="yes">C:\Program Files</directories>
  <directories check_all="yes" whodata="yes">C:\Program Files (x86)</directories>
  <directories check_all="yes" whodata="yes"
restrict="\.exe$|\.dll$">C:\Users\Public</directories>
  <windows_registry
arch="both">HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run</windows_regist
ry>
</syscheck>
```

- **D:\Shares\Finance** is the chapter's File Server scenario: `whodata` and `report_changes` deliver user, process and content diff.
- **Program Files and Program Files (x86)** are common persistence targets. `check_all` and `whodata` are worthwhile, without `report_changes`, since a binary diff has no reading value.
- **C:\Users\Public with restrict** limits the alert to executables and DLLs dropped in a classic staging location for lateral movement, without temporary-file noise.
- **The Registry Run key** is included because FIM natively covers the Windows Registry, and this is one of the highest-value persistence locations.

3.2 SACL and Audit Policy Auto-Configuration

When `whodata="yes"` is declared for a directory and the agent service restarts, Wazuh does not just start listening: it actively modifies the operating system. It applies a System Access Control List (SACL) to the monitored directory and enables the audit subcategories needed to generate the corresponding entries in the security log. Under normal conditions, on a standalone machine, this is automatic and requires no `auditpol` or `secpol.msc`.

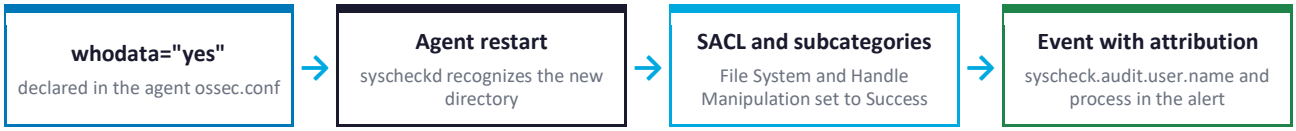


Figure 7 - whodata activation chain on Windows: from declaration to user attribution

Two Advanced Audit Policy subcategories, both under Object Access, are required, and the Wazuh documentation is explicit about the need for both.

Subcategory	GUID	What it covers
File System (Sistema de Arquivos on Portuguese systems)	`{0CCE921D-69AE-11D9-BED3-505054503030}`	Object access event when a handle is granted, based on the object's SACL

Subcategory	GUID	What it covers
Handle Manipulation (Manipulação de Identificador on Portuguese systems)	`{0CCE9223-69AE-11D9-BED3-505054503030}`	Handle close or duplication; carries the information that attributes file deletion

Table 7 - Audit subcategories required by whodata on Windows

MOST COMMON SYMPTOM: With only File System enabled, whodata reliably captures creation and modification and fails to attribute, or completely misses, deletion. Missing deletions alongside normal creations and modifications is the symptom most reported in the community, and an unconfigured Handle Manipulation is the most common root cause.

3.3 Validation with auditpol

The effective state of these subcategories must never be validated by reading the `secpol.msc` console. The Advanced Audit Policy Configuration node reflects the local GPO template on disk, not necessarily the effective policy in force. When a subcategory is set through the Windows audit policy API, exactly the mechanism of Wazuh auto-configuration, the change takes effect immediately in the effective policy, while `secpol.msc` may keep showing Not Configured. The reliable way to confirm the effective state is `auditpol`, querying by GUID to avoid failures on non-English Windows installations.

```
auditpol /get /subcategory:"{0CCE9223-69AE-11D9-BED3-505054503030}","{0CCE921D-69AE-11D9-BED3-505054503030}"
```

The first GUID corresponds to Handle Manipulation and the second to File System. Both must return Success (shown as Êxito on Portuguese systems, as in Figure 8). In PowerShell, the GUID must be enclosed in quotes, because PowerShell interprets the braces as the start of a script block and the command fails with a generic incorrect-parameter error unrelated to the audit policy.

Operating system language: Subcategory names vary with the language. On Brazilian Portuguese Windows they appear as "Manipulação de Identificador" and "Sistema de Arquivos"; in English, as "Handle Manipulation" and "File System". If `auditpol` fails with the name, always use the query by GUID.

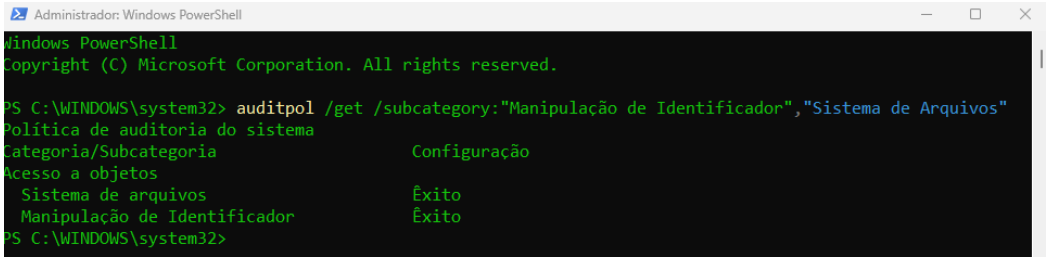


Figure 8 - Real evidence: auditpol confirming Success (Êxito) on File System (Sistema de Arquivos) and Handle Manipulation (Manipulação de Identificador) on LABDESK

Lab note: the capture of the "before" state of the whodata policy was not taken on LABDESK so as not to undo the configuration that was already validated. In production, capture the initial state before any change as a documented baseline for audit and rollback.

3.4 Local Policy versus GPO in a Domain-Joined Environment

On a domain-joined server, local auto-configuration is not the end of the story. If an Advanced Audit Policy GPO is linked to the File Server's organizational unit and explicitly defines File System or Handle Manipulation, that

GPO is reapplied on every Group Policy refresh cycle, by default every 90 to 120 minutes. If it sets those subcategories to a value different from what Wazuh needs, or simply resets them, each refresh silently reverts the auto-configuration, and deletion attribution fails again hours after it was confirmed, with no visible error in the Wazuh stack. If the GPO leaves both subcategories as Not Configured, it does not interfere.

To determine which scenario applies, run `gpresult /h report.html` on the server and inspect the Object Access section for the two subcategories, noting which GPO, if any, appears as the winning policy. If a conflicting GPO wins, the durable fix is to update it to explicitly set both subcategories to Success on the relevant OU.

> **Validation limit:** The `gpresult` evidence requires a domain-joined File Server. LABDESK operates in a workgroup, with no Active Directory binding. The behavior described above is documented based on the official Wazuh documentation and community records, and was not reproduced in this lab.

3.5 windows_audit_interval in Depth

The `windows_audit_interval` parameter is present in the default `ossec.conf` and is ignored by most deployments because its name does not suggest its function. It does not control the FIM scan frequency: it controls how often Wazuh checks whether the SACL of the monitored directory and the audit subcategories are still correctly configured.

```
<windows_audit_interval>60</windows_audit_interval>
```

The official documentation describes two mechanisms for detecting an external reversion of the audit configuration: monitoring EventID 4719, generated when an audit policy is modified, and `windows_audit_interval` itself, which periodically checks the audit policies and the SACLs of the monitored directories. When either mechanism detects a divergence, FIM switches all whodata directories to realtime, keeping detection without attribution. The official documentation does not describe automatic reapplication of the policy.



Figure 9 - Layered defense of the audit policy: reversion detection and fallback to realtime

> **LAB-VALIDATED LIMIT:** after a manager restart, whodata on LABDESK produced no syscheck events for about 25 minutes, until a manual `Restart-Service WazuhSvc` reapplied the SACL. `windows_audit_interval` only checks the policy and the SACLs and, on divergence, switches the directories to realtime; it does not reapply the SACL. Details in Finding AO-7 (chapter 10).

The default value is 300 seconds, which may be inadequate where GPO refresh is frequent or the risk of deliberate tampering is high. The value of 60 used in the lab can be kept in high-sensitivity environments, bearing in mind that each check queries the Windows API, with minimal but non-zero CPU cost at scale.

3.6 Rename as Deletion plus Creation

There is no native "renamed" event type. A rename is reported as two operations: deletion of the old path (rule ID 553) and creation of the new path (rule ID 554), with the same timestamp and the same user attribution

when whodata is active. If a requirement demands that renames appear as a single action, the correlation must be a custom rule that matches a 553 event and a 554 event with the same user and a short time window.

3.7 Windows Registry Monitoring

Wazuh FIM also covers the Windows Registry through dedicated decoders and rules, mapped to MITRE T1112 (Modify Registry). Malware that creates an executable in a monitored directory and registers a Run key to persist generates two correlatable alerts: a 554 (file added) and a 752 (registry value added).

Rule ID	Target	Event	MITRE
594	Key	Registry Key Integrity Checksum Changed	T1565.001, T1112
597	Key	Registry Key Entry Deleted	T1070.004, T1485, T1112
598	Key	Registry Key Entry Added	T1112
750	Value	Registry Value Integrity Checksum Changed	T1565.001, T1112
751	Value	Registry Value Entry Deleted	T1070.004, T1485, T1112
752	Value	Registry Value Entry Added	T1112

Table 8 - Native Windows Registry FIM rules

```
<windows_registry
arch="both">HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run</windows_regist
ry>
<windows_registry
arch="both">HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\RunOnce</windows_re
gistry>
<windows_registry
arch="both">HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services</windows_registry>
<windows_registry arch="both">HKEY_LOCAL_MACHINE\Software\Microsoft\Windows
NT\CurrentVersion\Winlogon</windows_registry>
```

The `arch="both"` attribute monitors the 32-bit and 64-bit views of the Registry, which matters because 32-bit malware on 64-bit systems can write to locations virtualized by WoW64 that do not appear in the default 64-bit view.

3.8 Lab Evidence: Full Cycle

To validate the cycle, create, modify and delete a test file in the directory declared with whodata and capture the result in Discover, filtering by `agent.name` and `rule.group:syscheck`. With correct configuration and auditing, the result is the three events, added, modified and deleted, with rule.id 554, 550 and 553, the correct path and `syscheck.audit.user.name` populated in all three.

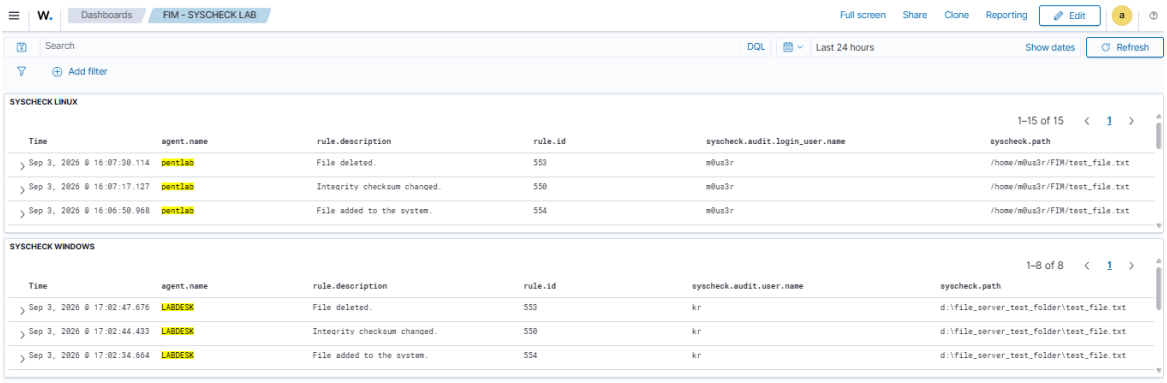


Figure 10 - Real evidence: FIM - SYSCHECK LAB dashboard comparing syscheck events on the Linux agent (pentlab) and on the Windows agent (LABDESK), with user attribution on both

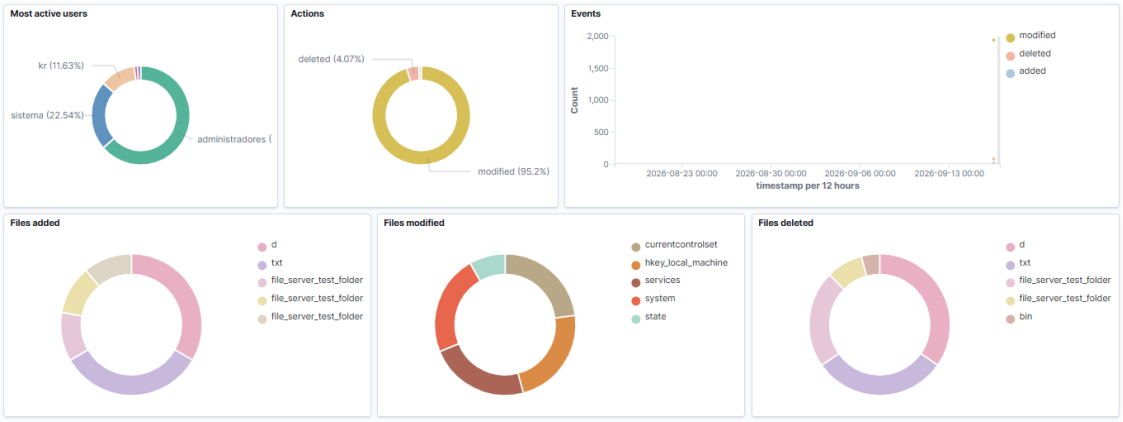


Figure 11 - Real evidence: FIM dashboard for the LABDESK agent (last 30 days) with event distribution by type: modified, deleted and added

4. FIM on Linux

This chapter covers FIM on Linux endpoints, focusing on the mechanism equivalent to Windows whodata, the Linux Audit subsystem (auditd), and on the eBPF-based alternative available since version 4.12.0.

4.1 Examples of Monitored Directories

```
<syscheck>
  <directories check_all="yes"
whodata="yes">/etc,/usr/bin,/usr/sbin,/bin,/sbin,/boot,/root</directories>
  <directories check_all="yes" whodata="yes"
report_changes="yes">/var/www/html</directories>
  <directories check_all="yes"
whodata="yes">/etc/passwd,/etc/shadow,/etc/sudoers</directories>
</syscheck>
```

The first line covers the classic baseline of system binaries and configuration files recommended for any hardened Linux host. The second targets a web root with content diffing, useful for catching webshells and defacement. The third isolates the three files most targeted for privilege escalation and persistence, justifying a line of its own so that a custom rule can alert on them at a higher severity than a generic change in `/etc`.

4.2 Installing and Configuring auditd

When `whodata="yes"` is declared for a directory, Wazuh automatically creates the corresponding audit rules in the kernel and, by default, restarts auditd to apply them. If auditd is not installed, on Debian or Ubuntu:

```
apt install -y auditd
```

On RHEL, CentOS or AlmaLinux, use `yum` or `dnf install auditd`, depending on the version. For Audit version 3.1.1 or later, you must also install the `audispd-plugins` package, which provides the `audisp-af_unix` plugin, responsible for delivering events to the Wazuh socket, and restart auditd to load the plugin.

```
yum install audispd-plugins
systemctl restart auditd
```

Two settings must exist on the endpoint. First, the agent reads the audit log itself:

```
<localfile>
  <log_format>audit</log_format>
  <location>/var/log/audit/audit.log</location>
</localfile>
```

Second, the auditd restart behavior is controlled by `restart_audit` inside the `whodata` block:

```
<whodata>
  <provider>audit</provider>
  <restart_audit>yes</restart_audit>
  <audit_key>auditkey1,auditkey2</audit_key>
  <startup_healthcheck>yes</startup_healthcheck>
  <queue_size>100000</queue_size>
</whodata>
```

RESTART_AUDIT=NO: Setting `restart_audit` to `no` means new whodata rules are not applied automatically: the Linux equivalent of a Windows machine where the SACL is never applied. whodata remains declared in `ossec.conf`, but the audit subsystem underneath is never configured to sustain it.

4.3 Manual Audit Rules and Persistence

To monitor paths outside FIM whose access still needs to be tracked, the `auditctl` syntax is:

```
auditctl -w /etc -p wa -k wazuh_fim
auditctl -w /usr/bin -p wa -k wazuh_fim
auditctl -w /usr/sbin -p wa -k wazuh_fim
auditctl -w /root -p wa -k wazuh_fim
```

The `-w` flag names the path, `-p wa` restricts the rule to writes and attribute changes (the combination relevant to FIM) and `-k` attaches a searchable key to each generated event, as Wazuh does in the rules it creates on its own. Rules created via `auditctl` live only in kernel memory and do not survive a reboot. To persist them, write the same rules to `/etc/audit/rules.d/`, for example `/etc/audit/rules.d/wazuh-fim.rules`, and load them with `augenrules --load`. A server that loses whodata attribution after a maintenance reboot, with the Wazuh configuration intact, almost certainly lost exactly that: rules that were created and never persisted.

User filters: -F auid

The most relevant filter for compliance- and investigation-oriented FIM is by `auid` (audit user ID), the UID of the login session, set at login and inherited by all child processes.

Goal	Rule	Effect
Root actions only	`auditctl -w /etc -p wa -F auid=0 -k wazuh_fim_root`	Captures events from root sessions; tracks suspicious administrative actions without system noise
Regular users	`auditctl -w /etc -p wa -F auid>=1000 -F auid!=4294967295 -k wazuh_fim_users`	Excludes the invalid loginuid (4294967295, equivalent to -1) of daemons without an interactive session
All sources	`auditctl -w /etc -p wa -k wazuh_fim_all`	No auid filter: includes system processes

Table 9 - auid filters in audit rules for FIM

> **Lab note:** With `whodata="yes"`, Wazuh generates the audit rules without the `-F auid` filter. Processes of the agent itself and system processes (journald, cron, apt) that write to `/etc` show up as whodata events. In noisy environments, manually add `-F auid>=1000` in `/etc/audit/rules.d/` after auto-configuration. Wazuh loads its wazuh_fim rules into the kernel when the agent starts; they are independent of the files in /etc/audit/rules.d/ (labnix: 4 persisted rules in /etc/audit/rules.d/net_connect.rules plus 6 wazuh_fim rules generated by the agent, one for each whodata directory: /etc, /usr/bin, /usr/sbin, /bin, /sbin and /boot).

4.4 The audit-keys CDB and Rule 80705

Raw Audit events are noisy. Wazuh uses the key argument to classify what generated each entry, through a CDB list on the server, `/var/ossec/etc/lists/audit-keys`.

```
audit-wazuh-w:write
audit-wazuh-r:read
audit-wazuh-a:attribute
audit-wazuh-x:execute
audit-wazuh-c:command
```

A custom key is added by including the line with the key used in `-k` and the desired category:

```
echo "wazuh_fim:write" >> /var/ossec/etc/lists/audit-keys
systemctl restart wazuh-manager
```

The default ruleset for Audit events is in `/var/ossec/ruleset/rules/0365-auditd_rules.xml`. Linux Audit alerts, unlike pure FIM alerts, appear in the Wazuh Dashboard Threat Hunting module and not in the File Integrity Monitoring module, a common point of confusion.

Rule ID **80705**, "Auditd: Configuration changed", is a diagnostic tool: it fires whenever the audit subsystem configuration changes, including the moment Wazuh applies its auto-configuration when the agent restarts. Filtering Threat Hunting by this ID at the restart time confirms that auto-configuration touched auditd, the Linux equivalent of Windows EventID 4719.

timestamp	agent.name	rule.description	rule.level	rule.id
Sep 3, 2026 @ 16:07:30.114	pentlab	File deleted.	7	553
Sep 3, 2026 @ 16:07:17.127	pentlab	Integrity checksum changed.	7	550
Sep 3, 2026 @ 16:06:50.968	pentlab	File added to the system.	5	554
Sep 3, 2026 @ 16:05:18.046	pentlab	Auditd: Configuration changed.	3	80705
Sep 3, 2026 @ 16:05:17.998	pentlab	Auditd: Configuration changed.	3	80705

Figure 12 - Real evidence: Threat Hunting on pentlab showing rule 80705 (Auditd: Configuration changed) at the moment of auto-configuration, alongside FIM events 554, 550 and 553

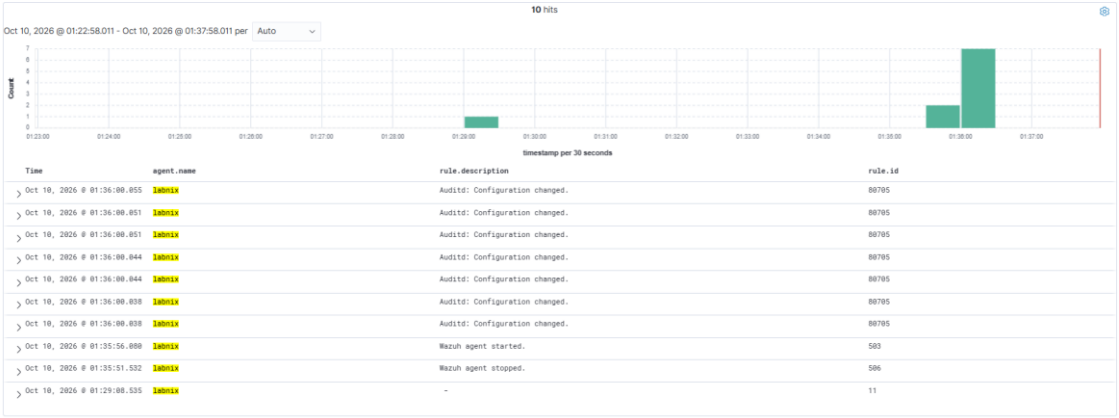


Figure 13 - labnix Discover, October 10, 2026: seven events of rule 80705 (Auditd: Configuration changed) at 01:36:00, right after the agent stop (rule 506, 01:35:51) and start (rule 503, 01:35:56)

4.5 eBPF Provider

Extended Berkeley Packet Filter (eBPF) safely runs programs in kernel space, without changing the kernel source code or loading modules. The eBPF mode of whodata, available since 4.12.0, extracts FIM events directly from the kernel through a `ring_buffer` structure, which hands them to user space where the FIM module processes them. The three tracked events map to kernel VFS operations:

Kernel hook	FIM event
`vfs_open`	File creation (file added)
`security_inode_setattr`	File modification (file modified)
`vfs_unlink`	File removal (file deleted)

Table 10 - Kernel hooks tracked by the eBPF provider

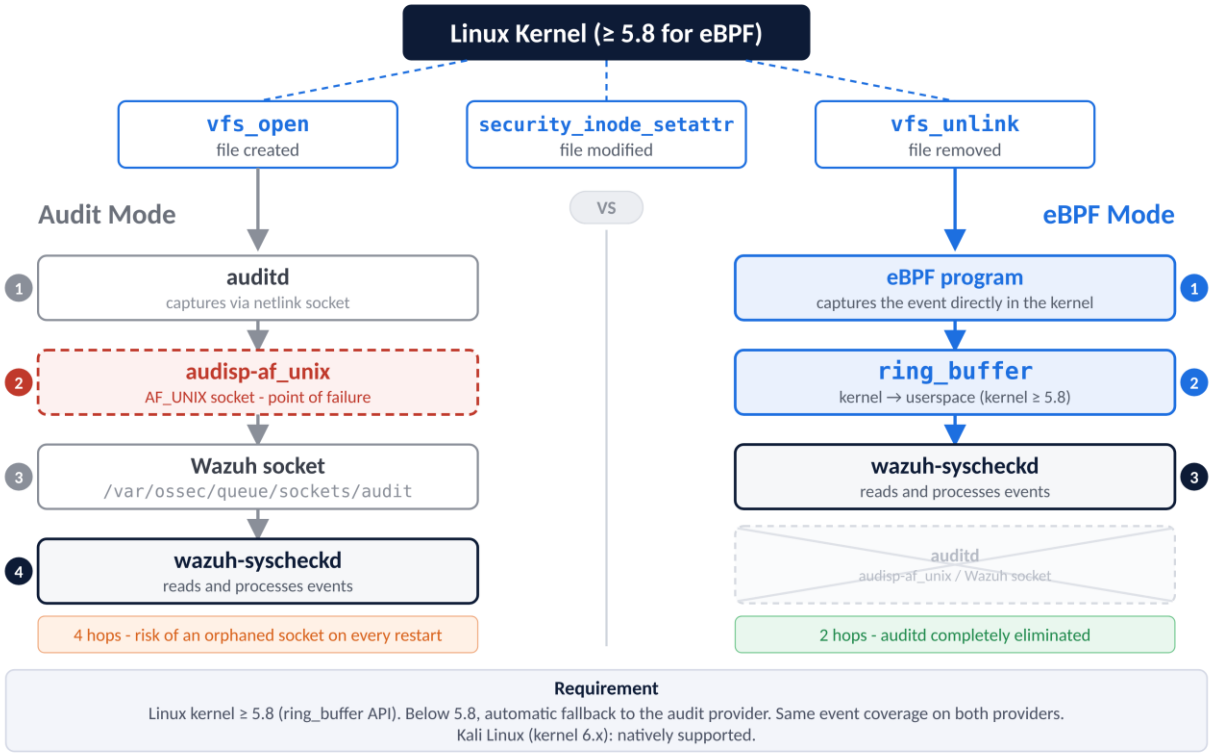


Figure 14 - Audit mode flow (4 steps) versus eBPF mode (3 steps): eBPF removes the dependency on auditd and the class of failures associated with it

The main difference is the elimination of auditd, which removes the associated class of failures, including the orphaned socket problem of section 4.6, with no cost in event coverage. The configuration is identical to audit, except for the `provider`:

```
<syscheck>
  <directories whodata="yes">/home/user/FIM</directories>
  <whodata>
    <provider>ebpf</provider>
    <queue_size>50000</queue_size>
  </whodata>
</syscheck>
```

- **Kernel requirement:** 5.8 or later, because the `ring_buffer` was introduced in that version. On earlier kernels, Wazuh automatically falls back to the audit provider.
- **queue_size:** eBPF detects events at kernel speed, without the auditd bottleneck. Bursts, such as mass deletion or backup, can exceed the default capacity (16384). Values of 50000 or more prevent loss, at the cost of more memory in the agent process.
- **Exclusive providers:** `audit` and `ebpf` are mutually exclusive per run; if both are declared, only the last one takes effect.
- **Fallback to audit:** a deployment configured as ebpf runs as audit when the eBPF health check fails, for example on kernels without BPF ring buffer and BTF. The agent logs (6959) and no alert is generated, which can happen on fleets with mixed kernels.

```
# Check the kernel version before setting the provider:
uname -r
# Log line for the started whodata engine (the string is "Whodata", without a hyphen):
wazuh-syscheckd: INFO: (6019): File integrity monitoring real-time Whodata engine started.
```

4.6 Common Failure: Orphaned audisp-af_unix Socket

A scenario reproduced in the lab deserves a section of its own because the symptom does not point to the cause. With auditd active and no apparent error in `systemctl status`, the service `journalctl` shows the `audisp-af_unix` plugin in a failure loop:

```
audisp-af_unix[NNN]: Couldn't bind af_unix socket (Address already in use)
audisp-af_unix[NNN]: audisp-af_unix plugin exiting due to errors setting up socket
auditd[NNN]: plugin /sbin/audisp-af_unix terminated unexpectedly
auditd[NNN]: plugin /sbin/audisp-af_unix has exceeded max_restarts
```

This plugin delivers audit events to the socket that Wazuh consumes at `/var/ossec/queue/sockets/audit`. With it dead, whodata does not work, even with the rules loaded in the kernel and `af_wazuh.conf` syntactically correct. On an AF_UNIX socket, "Address already in use" on bind normally does not indicate another live process: it indicates that the socket file already exists on disk, left over from a run that did not terminate cleanly.

```
ls -la /var/ossec/queue/sockets/audit
# If the file date is earlier than the last service restart:
systemctl stop auditd
rm -f /var/ossec/queue/sockets/audit
systemctl start auditd
```

The `journalctl` after the fix shows the plugin starting without error, with the dispatcher reporting one active plugin and no mention of bind or exceeded restart.

4.7 Validation and Lab Evidence

The startup health check exists because whodata on Linux depends on external state (loaded rules and an accessible socket) that can silently diverge. Disabling `startup_healthcheck` removes this protection and can leave the module in a degraded state without warning. A common failure signature in `ossec.log`: `syscheckd` reports a missing socket at `queue/sockets/audit`, tries to restart auditd, fails to reconnect and falls back to realtime with no alert, producing events without user or process attribution, as on Windows.

```
# Confirm that the Wazuh rules were loaded in the kernel:
auditctl -l | grep wazuh_fim
-w /home/user/FIM -p wa -k wazuh_fim
# Detect loss of audit events and enlarge the buffer:
auditctl -s | grep lost
auditctl -b 16384
```

An empty result from `auditctl -l` with whodata declared and the agent restarted points to a failure in the `restart_audit` path; check the health check messages in `ossec.log` before assuming a misconfiguration. When the dispatcher's internal queue fills up, some audit events are lost, and the next scheduled scan generates the missing alerts without the whodata information, which looks identical to a misconfiguration on the dashboard.

> **Troubleshooting detail:** Searching for "who-data", with a hyphen, in `ossec.log` returns no lines. The recorded string is "Whodata", without a hyphen. A badly calibrated `grep` leads to the wrong conclusion that the engine did not start.

Scheduled scan and whodata are not mutually exclusive. The real `ossec.log` excerpt below shows both activities side by side, independent: the baseline scan continues as periodic reconciliation even with real-time detection active.

```
2026/09/03 16:06:04 wazuh-syscheckd: INFO: FIM sync module started.
2026/09/03 16:06:04 wazuh-syscheckd: INFO: (6019): File integrity monitoring real-time
Whodata engine started.
2026/09/03 16:07:05 wazuh-syscheckd: INFO: (6008): File integrity monitoring scan started.
2026/09/03 16:08:00 wazuh-syscheckd: INFO: (6009): File integrity monitoring scan ended.
```

On Linux, user attribution appears in `syscheck.audit.login_user.name`, a different field name from the one used on Windows (`syscheck.audit.user.name`).

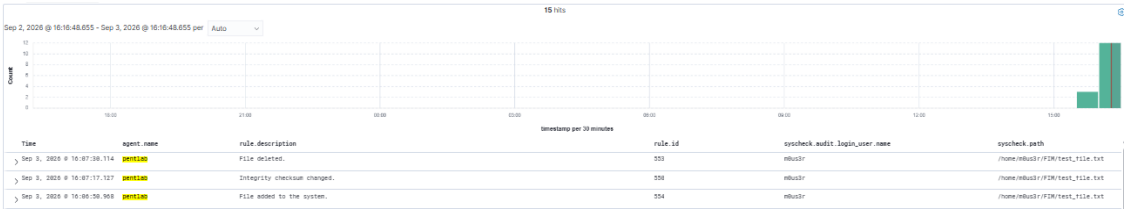


Figure 15 - Real evidence: pentlab agent Discover, /home/m0us3r/FIM/test_file.txt, with the three events of the cycle and attribution in syscheck.audit.login_user.name

4.8 inotify Watch Limit

This is a production issue for FIM on Linux that is easy to overlook, because the failure is silent. The kernel limits inotify watch instances per user, and the limit directly affects the realtime and whodata modes. Each directory with `realtime` or `whodata` consumes one instance; the default on most distributions is 8192. When the limit is reached, the kernel silently rejects new watches: the agent appears active, the scheduled scan works, but real-time events stop arriving for directories that exceeded the limit.

```
cat /proc/sys/fs/inotify/max_user_watches
cat /proc/$(pidof wazuh-agentd)/fdinfo/* 2>/dev/null | grep -c inotify
echo 'fs.inotify.max_user_watches=524288' >> /etc/sysctl.conf
sysctl -p
sysctl fs.inotify.max_user_watches
```

The value 524288 corresponds to 512 thousand watches and covers most production scenarios. The adjustment must be made before expanding the set of monitored directories, not after events stop, since the absence of events is the only visible evidence that the limit has been reached. Native rule 560, described in chapter 7, alerts when the kernel inotify queue overflows.

PATHS WITH /*: A path ending in `/*` in `<directories>` makes `syscheckd` expand the glob and register watches on existing individual files, with no active watch for new files. Declare the directory without a wildcard. Evidence in Finding AO-8 (chapter 10).

HOW TO CHECK THAT REALTIME DELIVERS EVENTS: Confirm on the manager, in `archives.json` or in the `wazuh-archives-*` index, that events arrive with `syscheck.mode` equal to `realtime`, even when the alert does not appear in `wazuh-alerts-*`: a child rule at level 0 suppresses the alert (section 5.2). On the agent, `strace -y -f -p` on `wazuh-syscheckd` should show `read()` on the `anon_inode:inotify` descriptor on every change; include `pselect6` in the syscall filter, since it is the wait call observed in `syscheckd`. Do not query `fim.db` while the agent is running: `sqlite3` returns `database is locked`.

4.9 Consolidated Troubleshooting Checklist

On both platforms, most reports of "FIM is not generating the expected events" are resolved by a short list of causes. Checking in this order is usually the fastest path.

1. Confirm that the path is declared in the effective `<syscheck>` block, locally or via group `agent.conf`, with the attributes the requirement demands (`check_all`, `whodata`, `report_changes`).
2. On Windows, confirm that the path is local, never UNC or a mapped drive. The restriction applies to every mode since 4.13.0.
3. Confirm that the agent service was restarted after the change. syscheckd does not apply new directories on a hot reload.
4. On Windows, validate the effective audit policy with `auditpol` by GUID, never through `secpol.msc`, and confirm Success on both subcategories.
5. On domain-joined Windows, run `gpresult /h` and look for a GPO that defines File System or Handle Manipulation.
6. On Linux, confirm that auditd is installed and active, the rules are present (`auditctl -l | grep wazuh_fim`) and `restart_audit` is not disabled.
7. On Linux, with auditd active and whodata inactive, look for "Address already in use" from `audisp-af_unix` in `journalctl` and remove the orphaned socket.
8. On Linux, check for lost events (`auditctl -s | grep lost`) and for rule persistence in `/etc/audit/rules.d/`.
9. On Linux, compare `max_user_watches` with the number of watches in use and check for paths without `/*`.
10. Review the agent `ossec.log` for the whodata health check and the fallback to realtime, searching for "Whodata" without a hyphen.

Figures 16 to 19 reorganize the checklist above as a decision tree for the question "why did the expected FIM event not arrive?". Figure 16 gathers the checks common to both platforms and routes to the figure for the symptom: Windows endpoint, Linux endpoint or manager. Each card gives the check, the answer that triggers the action and the reference to the section or finding in this handbook.

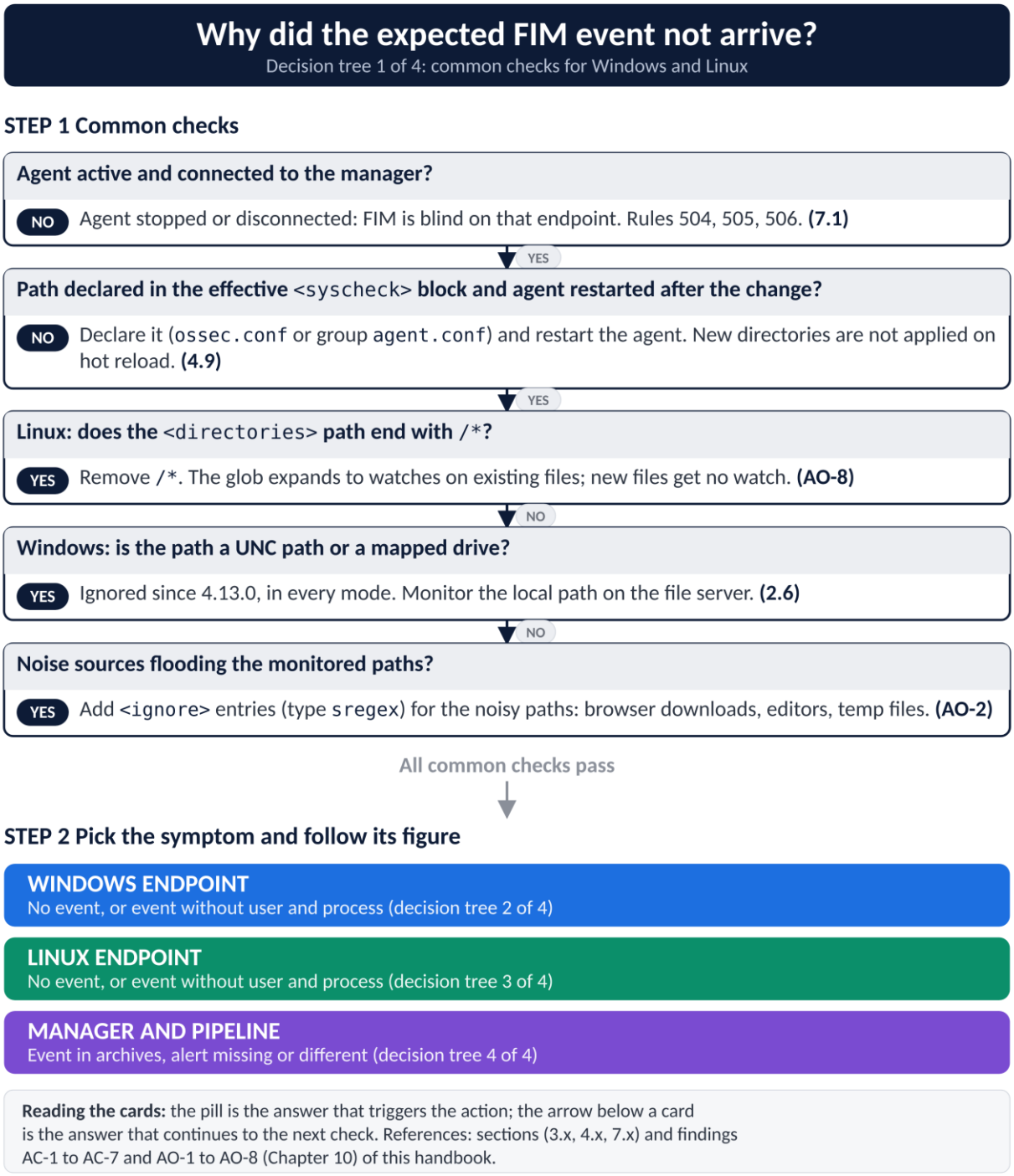


Figure 16 - Decision tree (1 of 4): checks common to both platforms and symptom selector

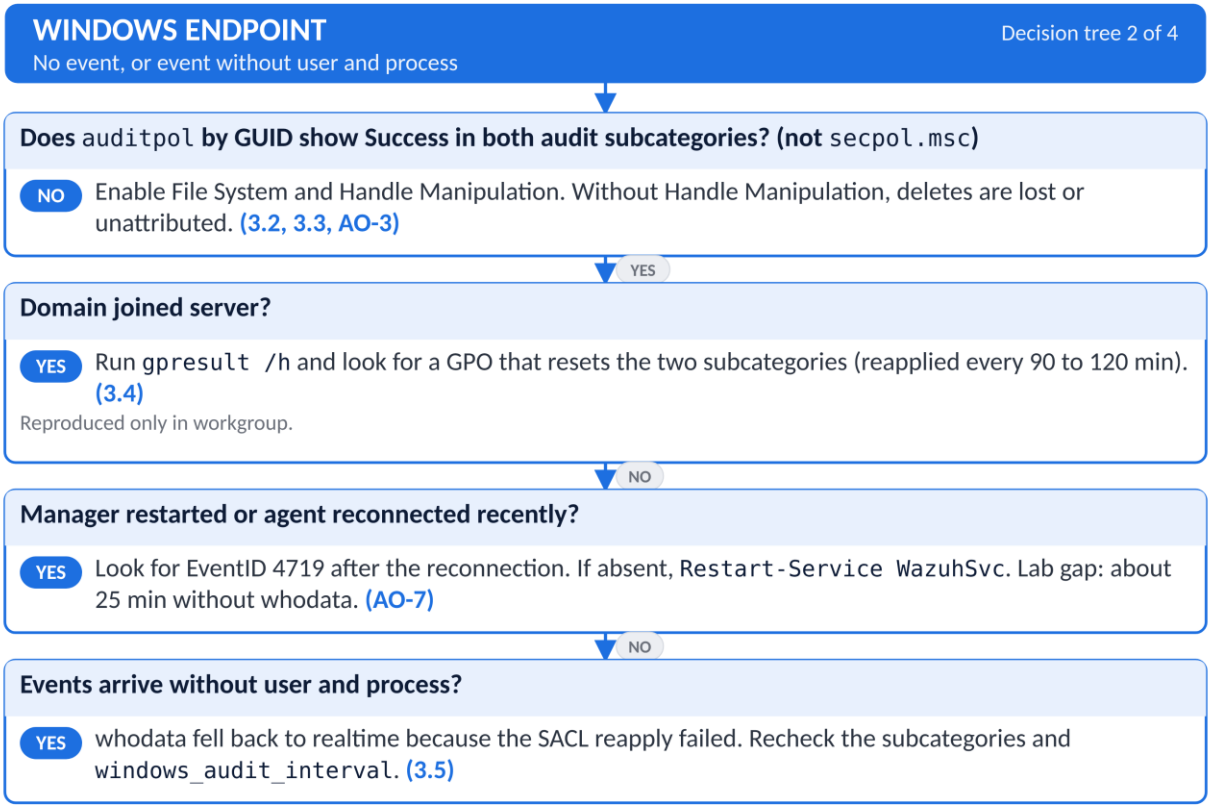


Figure 17 - Decision tree (2 of 4): Windows endpoint (audit policy, GPO, SACL after manager restart and whodata fallback)

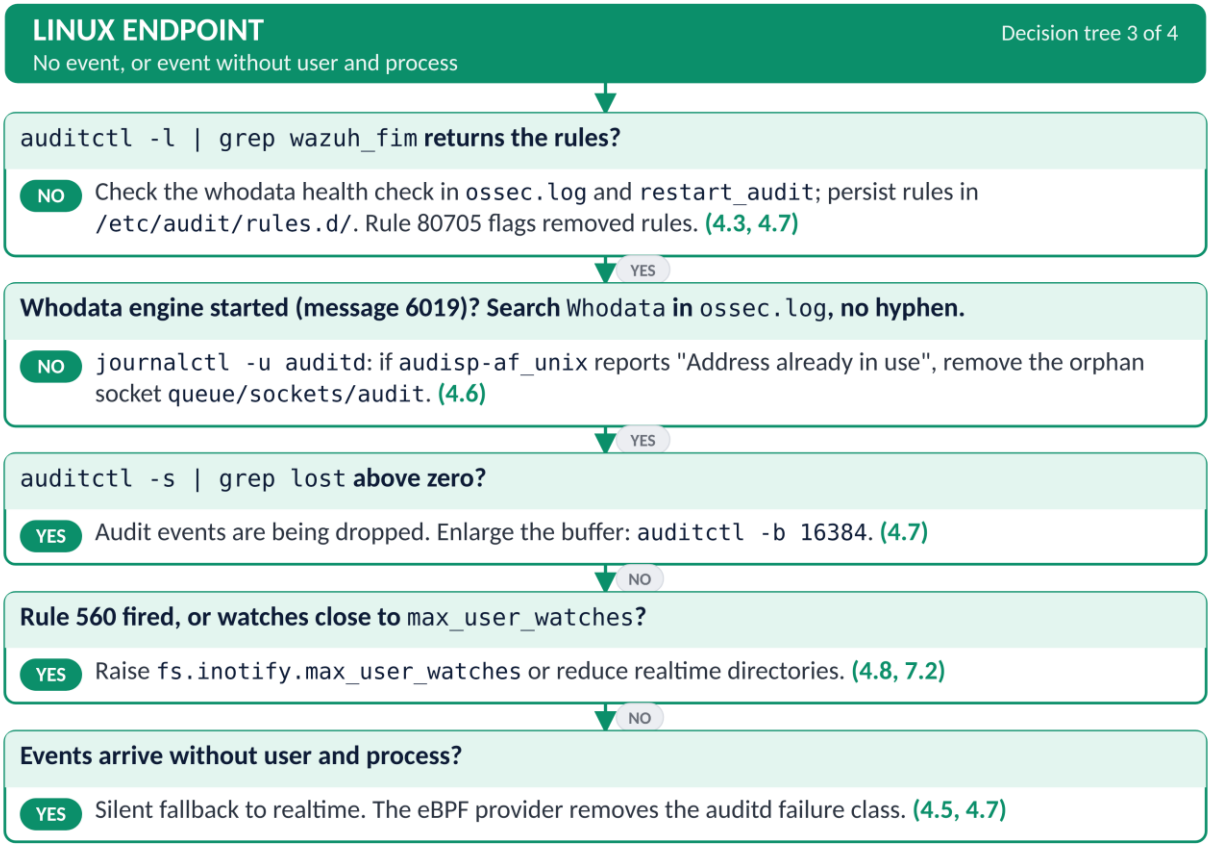


Figure 18 - Decision tree (3 of 4): Linux endpoint (audit rules, whodata engine, orphaned socket, lost events and inotify)

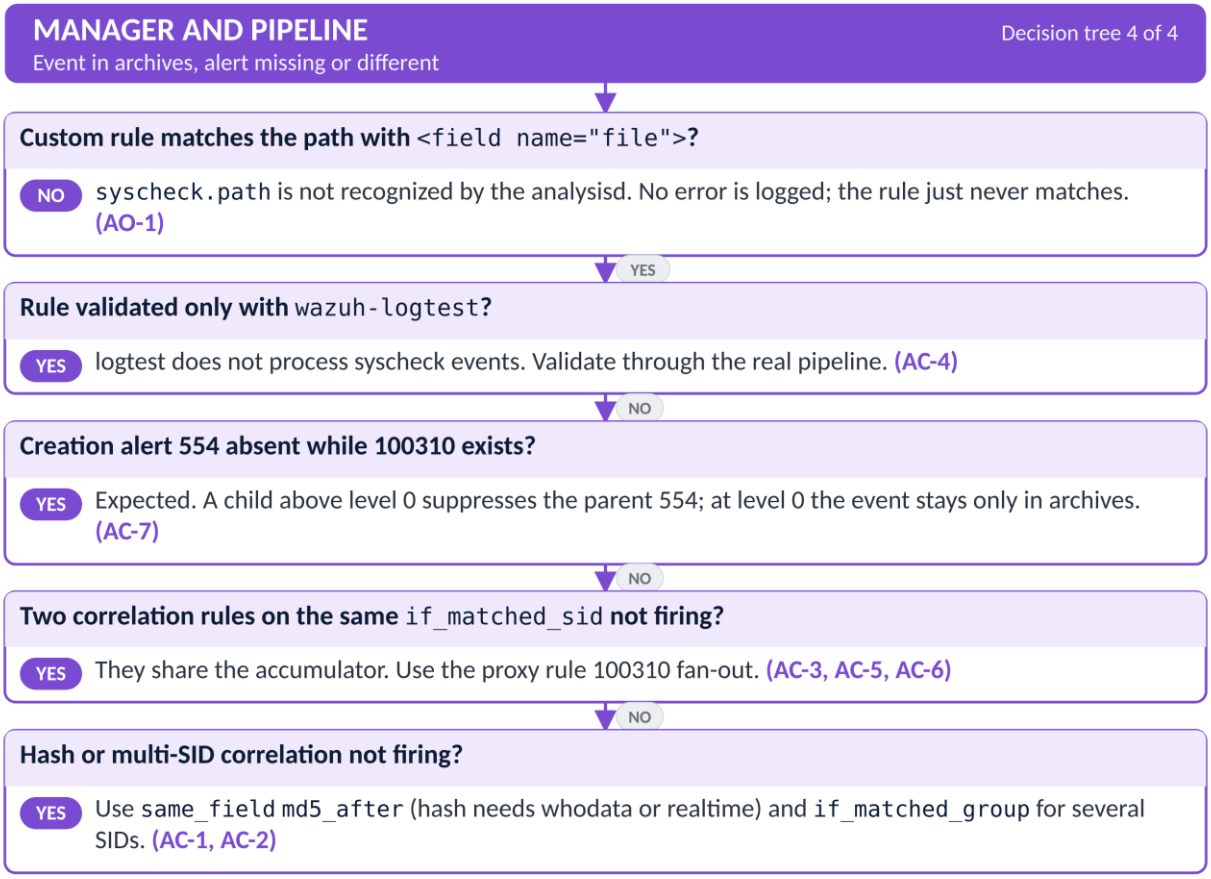


Figure 19 - Decision tree (4 of 4): manager and pipeline (rule field, logtest, proxy 100310 and correlation counters)

5. Anomaly Detection Rules

Native FIM detects individual events. Behavioral patterns, such as mass creation, mass deletion or replication of the same binary, require correlation. The six rules in this chapter, defined in `fim_anomaly_rules.xml`, extend base rules 550, 553 and 554 through temporal correlation and cover suspicious activity scenarios detectable exclusively via FIM, with no dependency on Sysmon or any additional telemetry.

THRESHOLD NOTICE: The `frequency="3"` and `timeframe="600"` (10 minutes) values were calibrated for lab testing. In production, especially on file servers, build hosts, backup systems and document repositories, tune them to the legitimate profile of each agent group before deployment. The `ignore="30"` is also a lab value; in production, `ignore="0"` guarantees coverage without alert suppression.

5.1 Rule Inventory

ID	Level	Type	Correlation	Trigger	MITRE
100310	5	Child/proxy	`if_sid` 554	Each `syscheck_new_entry` (554) is re-emitted to groups `fim_creation` and `fim_new_entry`	-
100300	10	Correlation	`if_matched_group fim_creation`	3 or more files created within 10 min on the same agent	T1565, T1565.001, T1074
100301	12	Correlation	`if_matched_group fim_new_entry`	Same MD5 (`md5_after`) in 3 or more directories within 10 min	T1105, T1570
100302	10	Correlation	`if_matched_sid` 550	3 or more files modified within 10 min	T1565, T1565.001
100303	10	Correlation	`if_matched_sid` 553	3 or more files deleted within 10 min	T1485, T1070, T1561
100304	14	Escalation	`if_matched_group syscheck_file`	5 or more mixed events (create, modify, delete) within 10 min	T1485, T1486, T1565

Table 11 - Custom FIM rules (file /var/ossec/etc/rules/fim_anomaly_rules.xml, permissions 660 wazuh:wazuh)

5.2 Counter Architecture and the 100310 Proxy Rule

Rule 100310 is the central architectural element. analysisd keeps correlation counters per base rule; when two correlation rules use `if_matched_sid` on the same SID, they share the accumulator and interfere with each other (Findings AC-3, AC-5 and AC-6). The solution is an application-level proxy: rule 100310, a child of 554, re-forwards each 554 event to two groups, creating two independent accumulators. It is the fan-out pattern of event-driven architectures.

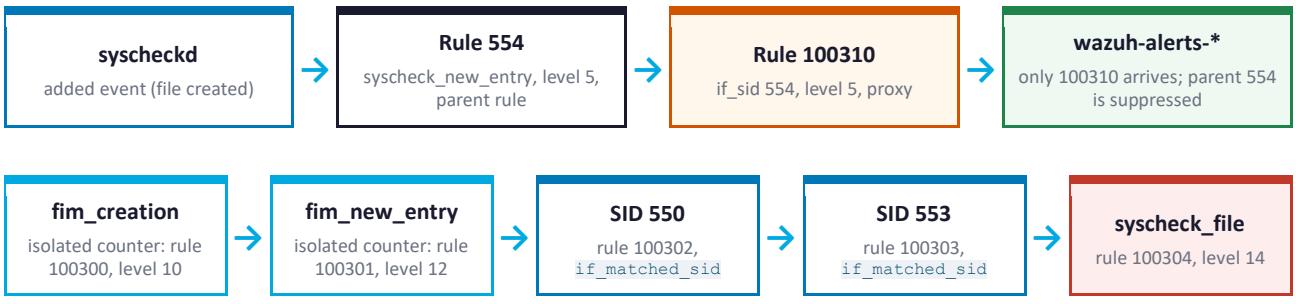


Figure 20 - Correlation flow: rule 100310 isolates the counters of 100300 and 100301; 100302 and 100303 correlate directly on 550 and 553

if_sid and if_matched_sid are distinct mechanisms: `<if_sid>` defines a child rule: it only fires if the specified SID has just fired on the same event. `<if_matched_sid>` defines temporal correlation: analysisd keeps an independent counter and fires when `frequency` events of the SID occur within the `timeframe`.

Rule 100310 uses `<if_sid>554` to intercept each 554 event and emit it to the `fim_creation` and `fim_new_entry` groups. Rules 100300 and 100301 use `<if_matched_group>` on those groups for independent counting. Without 100310, both would need `<if_matched_sid>554` and would share the counter.

CHILD RULE SIDE EFFECT (FINDING AC-7): With 100310 at a level greater than zero, analysisd suppresses the alert of the parent rule 554: in `wazuh-alerts-*` only `rule.id=100310` appears, never 554. At level 0, the alert is suppressed entirely and the event appears only in `wazuh-archives-*`, with no `rule` field. That is why 100310 operates at level 5 and the ID 100310 is listed in `FIM_FILE_RULES` of the investigation panel (chapter 6).

5.3 Rule Definitions

Current content of the file `fim_anomaly_rules.xml`, with 100310 at level 5 as validated on October 10, 2026. The descriptions remain in English, as in the deployed file.

```
<group name="syscheck,fim_anomaly,">


  <rule id="100310" level="5">
    <if_sid>554</if_sid>
    <description>FIM: New file detected (syscheck_new_entry) [base: rule 554]. Event forwarded to
fim_creation and fim_new_entry groups for independent counter isolation of rules 100300 and
100301.</description>
    <group>fim_creation,fim_new_entry,</group>
  </rule>


  <rule id="100300" level="10" frequency="3" timeframe="600" ignore="30">
    <if_matched_group>fim_creation</if_matched_group>
    <description>FIM: 3 or more files created within 10 minutes. Possible malware staging or automated
write activity. Lab threshold - tune for production.</description>
    <group>suspicious_creation,</group>
    <mitre><id>T1565</id><id>T1565.001</id><id>T1074</id></mitre>
  </rule>


  <rule id="100301" level="12" frequency="3" timeframe="600" ignore="30">
    <if_matched_group>fim_new_entry</if_matched_group>
    <same_field>md5_after</same_field>
    <different_field>file</different_field>
    <description>FIM: Same file hash created in 3 or more different directories within 10 minutes.
Possible malware propagation or dropper replication. Lab threshold - tune for production.</description>
    <group>suspicious_creation,</group>
    <mitre><id>T1105</id><id>T1570</id></mitre>
  </rule>


  <rule id="100302" level="10" frequency="3" timeframe="600" ignore="30">
    <if_matched_sid>550</if_matched_sid>
    <description>FIM: 3 or more files modified within 10 minutes. Possible payload execution, in-place
encryption, or automated tampering. Lab threshold - tune for production.</description>
    <group>suspicious_modification,</group>
    <mitre><id>T1565</id><id>T1565.001</id></mitre>
  </rule>


  <rule id="100303" level="10" frequency="3" timeframe="600" ignore="30">
    <if_matched_sid>553</if_matched_sid>
    <description>FIM: 3 or more files deleted within 10 minutes. Possible evidence destruction, anti-
forensics, or post-execution malware cleanup. Lab threshold - tune for production.</description>
    <group>suspicious_deletion,</group>
    <mitre><id>T1485</id><id>T1070</id><id>T1561</id></mitre>
  </rule>
```

```
  <rule id="100304" level="14" frequency="5" timeframe="600" ignore="30">
    <if_matched_group>syscheck_file</if_matched_group>
    <description>FIM: High volume mixed file activity - 5 or more create, modify, and delete events
within 10 minutes. Combined pattern suggesting active malware execution. Lab threshold - tune for
production.</description>
    <group>suspicious_combined,fim_escalation,</group>
    <mitre><id>T1485</id><id>T1486</id><id>T1565</id></mitre>
  </rule>

</group>
```

Deploy

```
chown wazuh:wazuh fim_anomaly_rules.xml
chmod 660 fim_anomaly_rules.xml
cp fim_anomaly_rules.xml /var/ossec/etc/rules/
/var/ossec/bin/wazuh-analysisd -t 2>&1 | grep -i error
systemctl restart wazuh-manager
```

Usage notes

- **Per-directory scope:** in deployments with several paths, restrict 100300 per directory with a `<field name="file" type="pcre2">(?i)/path/target</field>` filter. In custom FIM rules, the correct path field is `file`, not `syscheck.path` (Finding AO-1).
- **Hash in 100301:** the correlation field is `md5_after`, not `md5` (Finding AC-1).
- **Multiple SIDs:** 100304 uses `if_matched_group` on `syscheck_file`, the group of rules 550, 553 and 554 in `0015-ossec_rules.xml`, because `if_matched_sid` with comma-separated SIDs is not reliable (Finding AC-2).
- **Cooldown ignore:** in the lab, `ignore` proved to be global per rule, not per agent: a trigger on agent A suppressed the same rule on agent B within the period. With agents of different profiles, consider `ignore="0"` and control volume through agent groups and per-group thresholds.
- **Validation:** `wazuh-logtest` does not process syscheck events (Finding AC-4); always validate through the real pipeline.

5.4 Scenario 1: File Creation (554, 100310, 100300, 100301)

Base rule 554 (`syscheck_new_entry`, level 5) fires for each new file. 100310 replaces it in the alert pipeline (Finding AC-7). 100300 detects mass creation: it indicates malware staging, automated execution or mass writing of artifacts by a malicious process. 100301, the highest-level rule in the group, detects the same content (identical MD5) in three or more distinct directories: dropper replication or lateral tool staging.

The simulation scripts created four files with a 3-second interval between each operation, keeping the correlation thresholds within the 10-minute window.

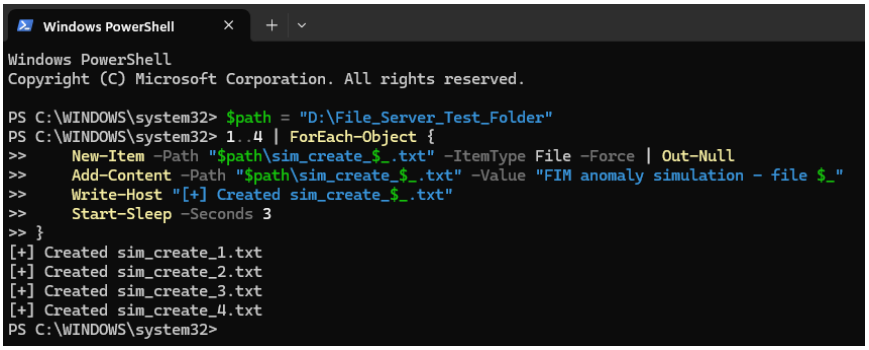


Figure 21 - PowerShell file creation simulation script on LABDESK (D:\File_Server_Test_Folder)

```
┌──(m0us3r㉿ pentlab)-[~]
└─$ for i in {1..4}; do
    echo "FIM anomaly simulation - file $i" > /home/m0us3r/FIM/sim_create_$i.txt
    echo "[+] Created sim_create_$i.txt"
    sleep 3
done
[+] Created sim_create_1.txt
[+] Created sim_create_2.txt
[+] Created sim_create_3.txt
[+] Created sim_create_4.txt
```

Figure 22 - Bash file creation simulation script on pentlab (/home/m0us3r/FIM)

30 hits

Sep 4, 2026 @ 08:31:02.016 - Sep 11, 2026 @ 08:31:02.016

Export Formatted | Reset view | 1273 available fields | Columns | Density | **1 fields sorted** | Full screen

timestamp	agent.name	syscheck.path	syscheck.event	rule.description	rule.level	rule.id
Sep 10, 2026 @ 22:25:09.367	LABDESK	d:\file_server_test_folder\t99_4.txt	added	File added to the system.	5	554
Sep 10, 2026 @ 22:25:05.338	LABDESK	d:\file_server_test_folder\t99_2.txt	added	File added to the system.	5	554
Sep 10, 2026 @ 22:25:03.333	LABDESK	d:\file_server_test_folder\t99_1.txt	added	File added to the system.	5	554
Sep 10, 2026 @ 22:18:11.611	LABDESK	d:\file_server_test_folder\tst4_6.txt	added	File added to the system.	5	554
Sep 10, 2026 @ 22:18:09.609	LABDESK	d:\file_server_test_folder\tst4_5.txt	added	File added to the system.	5	554

Figure 23 - Rule 554: 30 creation events (syscheck.event added, level 5) on LABDESK

28 hits

Sep 4, 2026 @ 08:41:01.287 - Sep 11, 2026 @ 08:41:01.287

Export Formatted | Reset view | 1273 available fields | Columns | Density | **1 fields sorted** | Full screen

timestamp	agent.name	syscheck.path	syscheck.event	rule.description	rule.level	rule.id
Sep 10, 2026 @ 19:13:40.876	pentlab	/home/m0us3r/FIM/esc2_2.txt	added	File added to the system.	5	554
Sep 10, 2026 @ 19:13:40.785	pentlab	/home/m0us3r/FIM/esc2_1.txt	added	File added to the system.	5	554
Sep 10, 2026 @ 19:10:12.099	pentlab	/home/m0us3r/FIM/esc_3.txt	added	File added to the system.	5	554
Sep 10, 2026 @ 19:10:12.099	pentlab	/home/m0us3r/FIM/esc_2.txt	added	File added to the system.	5	554
Sep 10, 2026 @ 19:09:22.047	pentlab	/home/m0us3r/FIM/mix_3.txt	added	File added to the system.	5	554

Figure 24 - Rule 554: 28 creation events on pentlab (Linux)



Figure 25 - Rule 554: 16 creation events on labnix (Linux, Ubuntu 24.04.5), 2026-10-09; the file p100301v3.txt was created at 20:51:17 in /var/tmp, /home/kr/Desktop, /home/kr/Downloads and /tmp, with Mode: realtime



Figure 26 - Rule 100310 (proxy, child of 554) with the description "[base: rule 554]" on labnix, 2026-10-10 13:26:17

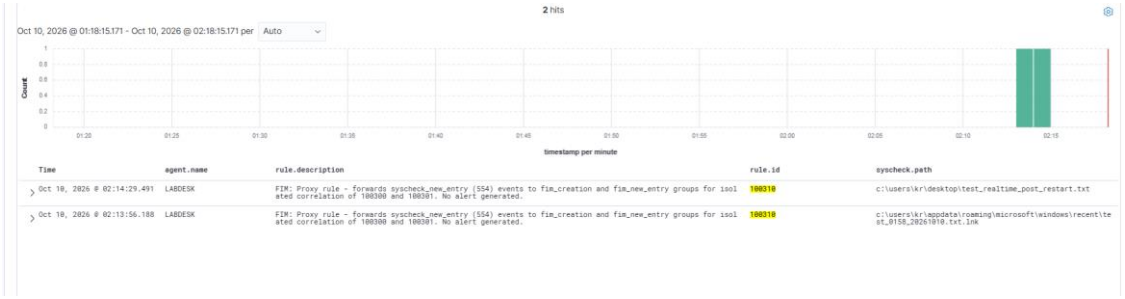


Figure 27 - Rule 100310 on LABDESK, 2026-10-10: creation of test_realtime_post_restart.txt on the Desktop (02:14:29) and of a .lnk shortcut in Recent (02:13:56); the description shown is the one prior to the update with [base: rule 554]



Figure 28 - Rule 100310 on labnix, 2026-10-10: the hits at 02:30:14 and 02:31:41 show the previous description ("Proxy rule ... No alert generated") and the one at 02:35:52 already shows the updated description ("New file detected (syscheck_new_entry)")

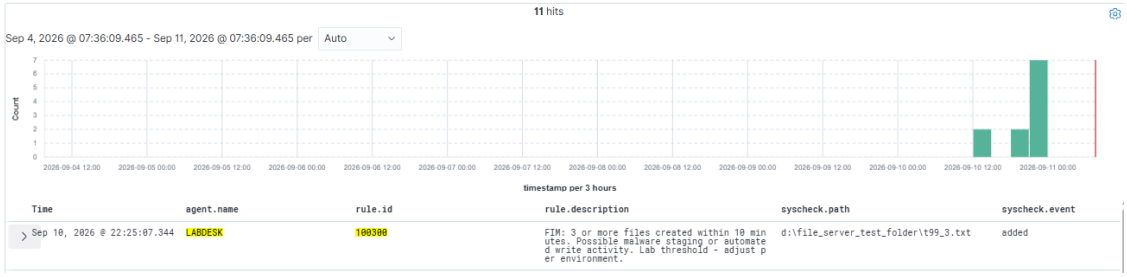


Figure 29 - Rule 100300 triggered on LABDESK (Windows), syscheck.event added

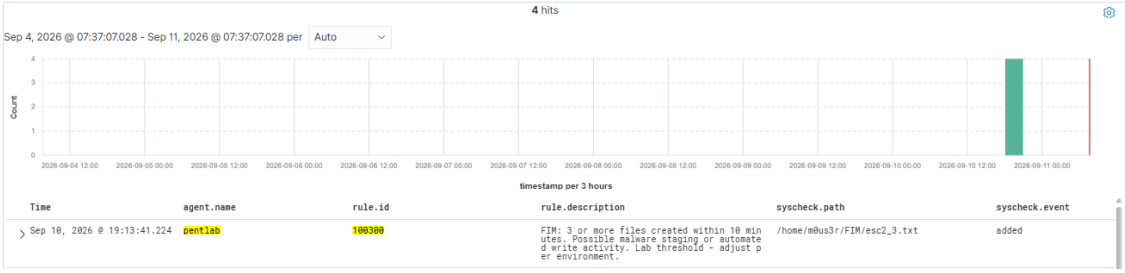


Figure 30 - Rule 100300 triggered on pentlab (Linux), syscheck.event added



Figure 31 - Rule 100300 triggered on labnix (Linux, Ubuntu 24.04.5): syscheck.event added at /var/tmp/payload_test.txt, 2026-10-09 20:18:07; 22 occurrences in 24 hours

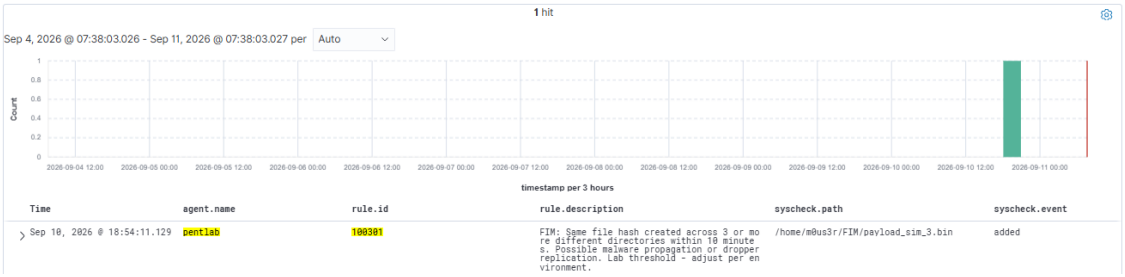


Figure 32 - Rule 100301 triggered on pentlab: same MD5 hash (md5_after) in distinct directories, file payload_sim_3.bin

5.5 Scenario 2: File Modification (550, 100302)

Base rule 550 (`syscheck_integrity_changed`, level 7) records the path and the hashes before and after the modification. 100302 fires with 3 or more modifications in 10 minutes and correlates directly on `if_matched_sid` 550, with no proxy rule, because no child rule of 550 interferes with the counter. It indicates payload execution, in-place encryption (the initial ransomware pattern) or automated configuration tampering. On Linux, with a `frequency` of 60 s, the scheduled scan may generate 550 events due to mtime changes on recently created files: expected behavior in the lab, and not in production with the default frequency of 43200 s.

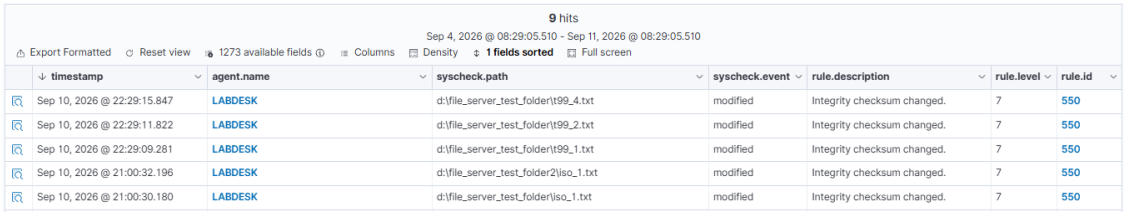


Figure 33 - Rule 550: 9 modification events (level 7) on LABDESK

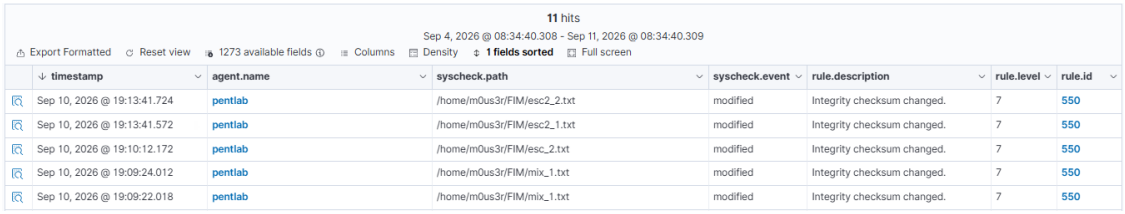


Figure 34 - Rule 550: 11 modification events on pentlab

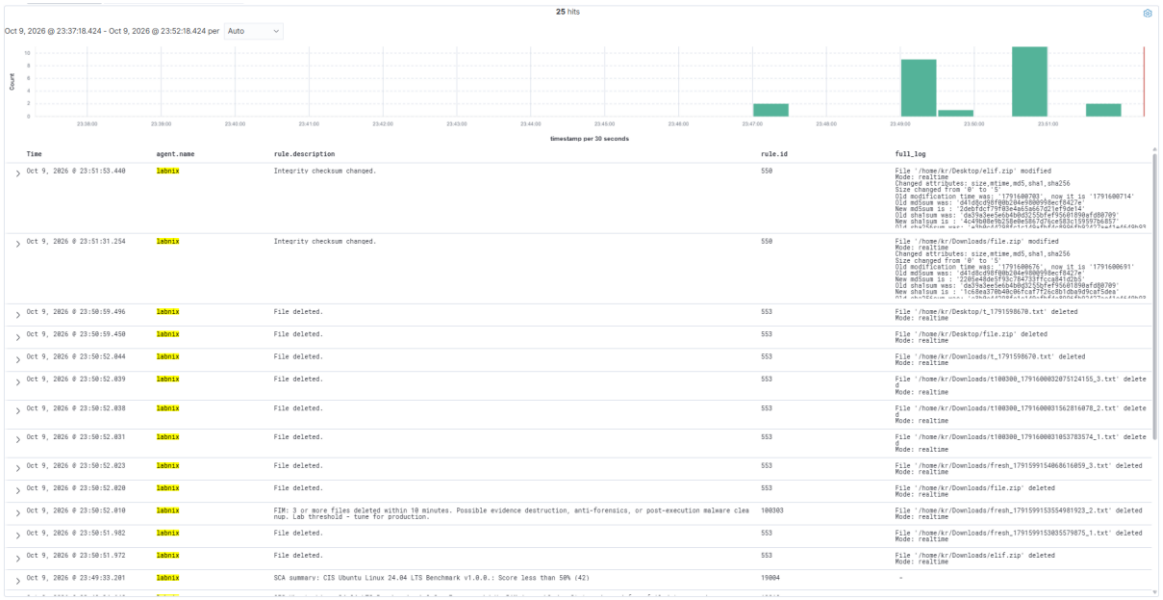


Figure 35 - Rule 550 on labnix, 2026-10-09 23:51: modifications of /home/kr/Desktop/elif.zip and /home/kr/Downloads/file.zip (Mode: realtime; size, mtime, md5, sha1 and sha256 changed), followed by the deletions in sequence (553); 25 hits

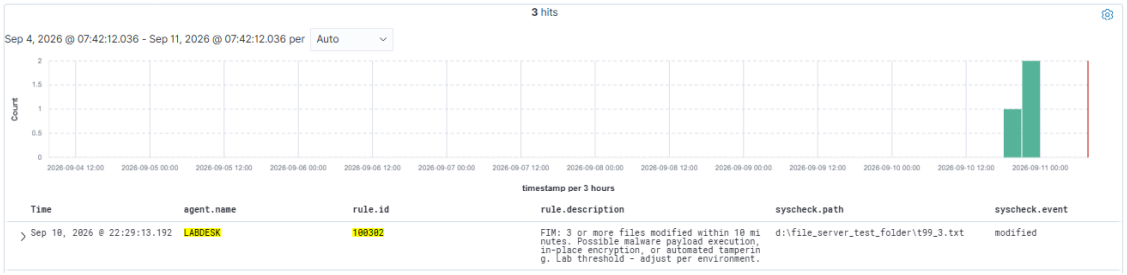


Figure 36 - Rule 100302 triggered on LABDESK, syscheck.event modified

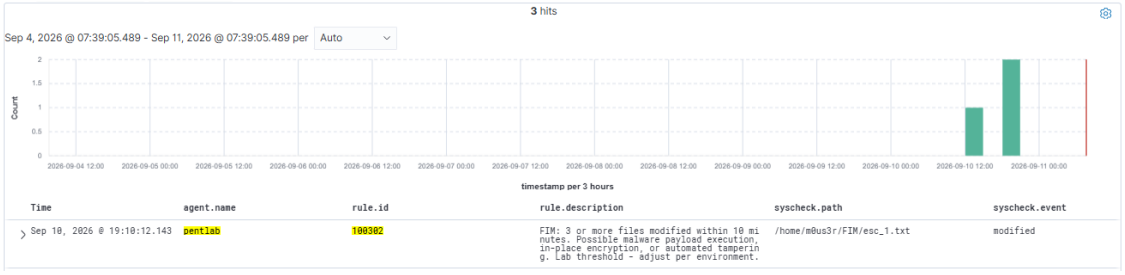


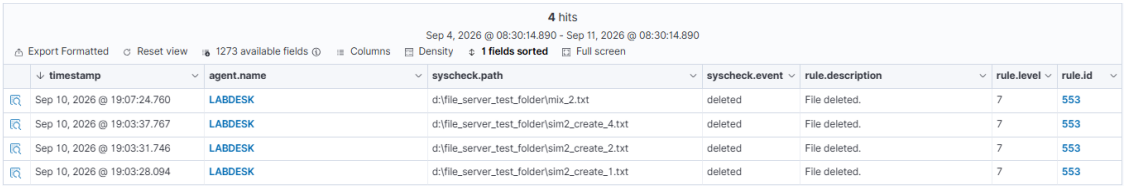
Figure 37 - Rule 100302 triggered on pentlab, syscheck.event modified



Figure 38 - Rule 100302 triggered on labnix (Linux, Ubuntu 24.04.5): /home/kr/Downloads/beacon_test.sh modified (Mode: realtime; size, md5, sha1 and sha256 changed), 2026-10-08 12:30:12

5.6 Scenario 3: File Deletion (553, 100303)

Base rule 553 (`syscheck_deleted`, level 7) records the deletion. 100303 fires with 3 or more deletions in 10 minutes and suffers no counter interference, because there is no child rule associated with SID 553. It indicates evidence destruction, anti-forensics, log removal or post-execution malware cleanup.



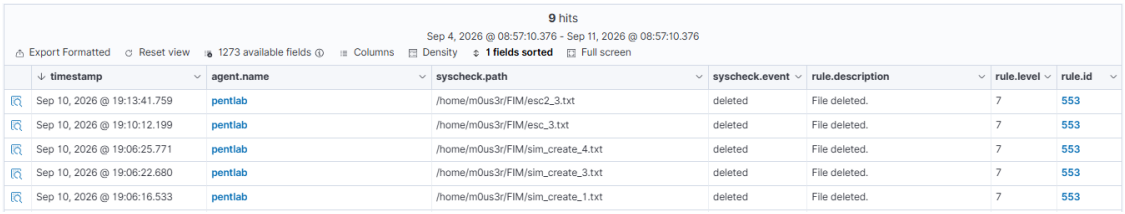
4 hits

Sep 4, 2026 @ 08:30:14.890 - Sep 11, 2026 @ 08:30:14.890

Export Formatted | Reset view | 1273 available fields | Columns | Density | 1 fields sorted | Full screen

timestamp	agent.name	syscheck.path	syscheck.event	rule.description	rule.level	rule.id
Sep 10, 2026 @ 19:07:24.760	LABDESK	d:\file_server_test_folder\mix_2.txt	deleted	File deleted.	7	553
Sep 10, 2026 @ 19:03:37.767	LABDESK	d:\file_server_test_folder\sim2_create_4.txt	deleted	File deleted.	7	553
Sep 10, 2026 @ 19:03:31.746	LABDESK	d:\file_server_test_folder\sim2_create_2.txt	deleted	File deleted.	7	553
Sep 10, 2026 @ 19:03:28.094	LABDESK	d:\file_server_test_folder\sim2_create_1.txt	deleted	File deleted.	7	553

Figure 39 - Rule 553: 4 deletion events (level 7) on LABDESK



9 hits

Sep 4, 2026 @ 08:57:10.376 - Sep 11, 2026 @ 08:57:10.376

Export Formatted | Reset view | 1273 available fields | Columns | Density | 1 fields sorted | Full screen

timestamp	agent.name	syscheck.path	syscheck.event	rule.description	rule.level	rule.id
Sep 10, 2026 @ 19:13:41.759	pentlab	/home/m0us3r/FIM/esc2_3.txt	deleted	File deleted.	7	553
Sep 10, 2026 @ 19:10:12.199	pentlab	/home/m0us3r/FIM/esc_3.txt	deleted	File deleted.	7	553
Sep 10, 2026 @ 19:06:25.771	pentlab	/home/m0us3r/FIM/sim_create_4.txt	deleted	File deleted.	7	553
Sep 10, 2026 @ 19:06:22.680	pentlab	/home/m0us3r/FIM/sim_create_3.txt	deleted	File deleted.	7	553
Sep 10, 2026 @ 19:06:16.533	pentlab	/home/m0us3r/FIM/sim_create_1.txt	deleted	File deleted.	7	553

Figure 40 - Rule 553: 9 deletion events on pentlab



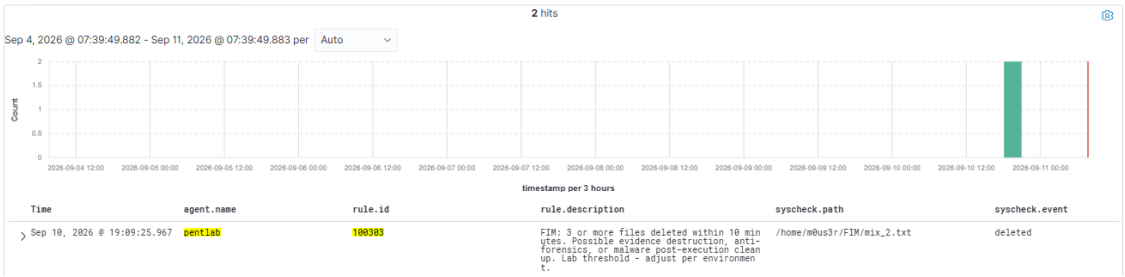
34 hits

Oct 9, 2026 @ 22:16:10.758 - Oct 9, 2026 @ 23:16:10.758 per Auto

timestamp per minute

Oct 9, 2026 @ 23:15:45.059	labnix	File deleted.	553	File '/home/kr/Desktop/file.zip' deleted Mode: realtime
Oct 9, 2026 @ 23:15:21.014	labnix	File deleted.	553	File '/home/kr/Downloads/t_1791597370.txt' deleted Mode: realtime
Oct 9, 2026 @ 23:15:21.005	labnix	File deleted.	553	File '/home/kr/Downloads/TESTING.txt' deleted Mode: realtime
Oct 9, 2026 @ 23:15:21.002	labnix	File deleted.	553	File '/home/kr/Downloads/t_1791591307.txt' deleted Mode: realtime
Oct 9, 2026 @ 23:15:21.000	labnix	File deleted.	553	File '/home/kr/Downloads/payload_test.txt' deleted Mode: realtime
Oct 9, 2026 @ 23:15:20.996	labnix	File deleted.	553	File '/home/kr/Downloads/p100301v4.txt' deleted Mode: realtime
Oct 9, 2026 @ 23:15:20.989	labnix	File deleted.	553	File '/home/kr/Downloads/p5.txt' deleted Mode: realtime
Oct 9, 2026 @ 23:15:20.976	labnix	File deleted.	553	File '/home/kr/Downloads/p100301.txt' deleted Mode: realtime
Oct 9, 2026 @ 23:15:20.976	labnix	File deleted.	553	File '/home/kr/Downloads/p100301v3.txt' deleted Mode: realtime
Oct 9, 2026 @ 23:15:16.127	labnix	FIM: 3 or more files deleted within 10 minutes. Possible evidence destruction, anti-forensics, or post-execution malware cleanup. Lab threshold - tune for production.	100303	File '/home/kr/Downloads/file.zip' deleted Mode: realtime
Oct 9, 2026 @ 23:08:33.257	labnix	File deleted.	553	File '/tmp/tmp.W1S71uGJAV' deleted Mode: realtime
Oct 9, 2026 @ 23:08:33.256	labnix	FIM: 3 or more files deleted within 10 minutes. Possible evidence destruction, anti-forensics, or post-execution malware cleanup. Lab threshold - tune for production.	100303	File '/tmp/tmp.r5QKE8X2Tz' deleted Mode: realtime
Oct 9, 2026 @ 23:08:33.256	labnix	File deleted.	553	File '/tmp/tmp.V4ey0ZXe1t' deleted Mode: realtime
Oct 9, 2026 @ 23:08:33.230	labnix	Integrity checksum changed.	550	File '/tmp/tmp.V4ey0ZXe1t' modified Mode: realtime Changed attributes: size,mtime,md5,sha1,sha256 Size changed from '0' to '100' Old modification time was: '1791598112', now it is '179159 8113' Old md5sum was: 'd41d8cd98f00b204e9800998ecf8427e'

Figure 41 - Rule 553 on labnix, 2026-10-09 from 23:08 to 23:15: deletions in /home/kr/Downloads, /home/kr/Desktop and /tmp (Mode: realtime), with 100303 firing at 23:08:33 and at 23:15:16; 34 hits



2 hits

Sep 4, 2026 @ 07:39:49.882 - Sep 11, 2026 @ 07:39:49.883 per Auto

timestamp per 3 hours

Time	agent.name	rule.id	rule.description	syscheck.path	syscheck.event
Sep 10, 2026 @ 19:09:25.967	pentlab	100303	FIM: 3 or more files deleted within 10 minutes. Possible evidence destruction, anti-forensics, or malware post-execution clean up. Lab threshold - adjust per environmen t.	/home/m0us3r/FIM/mix_2.txt	deleted

Figure 42 - Rule 100303 triggered on pentlab, syscheck.event deleted

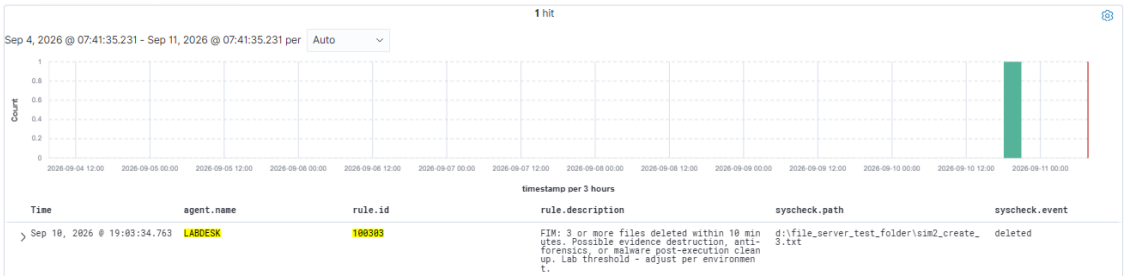


Figure 43 - Rule 100303 triggered on LABDESK, syscheck.event deleted

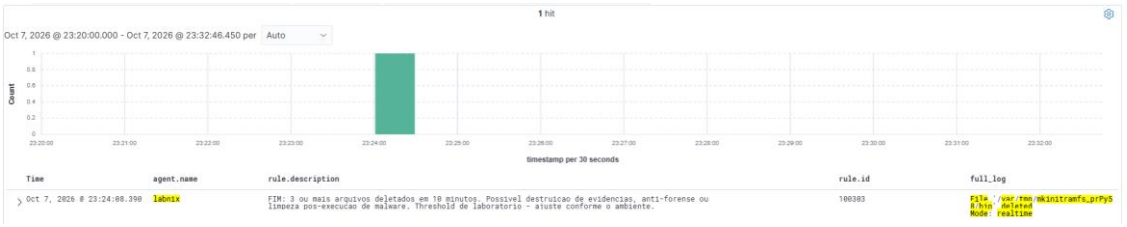


Figure 44 - Rule 100303 triggered on labnix (Linux, Ubuntu 24.04.5): deletion of /var/tmp/mkinitramfs_prPy58/bin (Mode: realtime), 2026-10-07 23:24:08

5.7 Escalation: High-Volume Mixed Activity (100304)

100304 is the escalation rule, level 14, and demands immediate attention. It fires with 5 or more events of any combination of creation, modification and deletion in 10 minutes on the same agent. The create-modify-delete or write-then-delete pattern is a strong behavioral indicator of active malware execution.

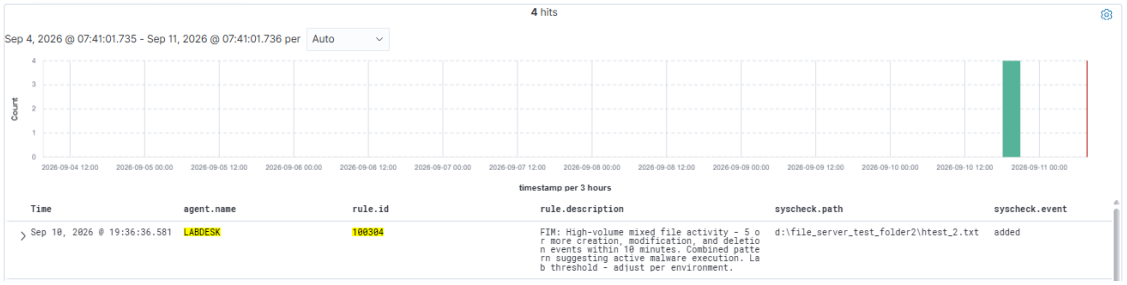


Figure 45 - Rule 100304 (level 14) triggered on LABDESK: mixed activity within 10 minutes

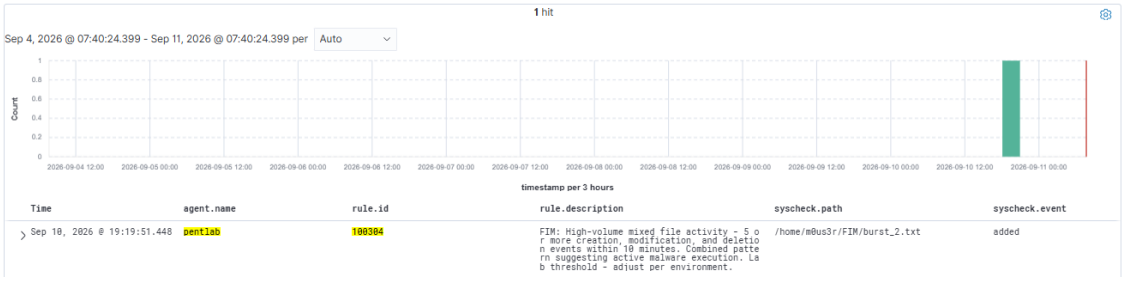


Figure 46 - Rule 100304 (level 14) triggered on pentlab

5.8 Validation Matrix

Rule	Scenario	LABDESK (Windows)	pentlab (Linux)	Observation
100310	Isolation proxy (level 5)	**CONFIRMED**	-	Validated on LABDESK and labnix (Linux) on 2026-10-10

Rule	Scenario	LABDESK (Windows)	pentlab (Linux)	Observation
100300	3+ creations within 10 min	CONFIRMED	CONFIRMED	Uses `fim_creation` via 100310
100301	Same hash, different directories	PARTIAL	CONFIRMED	Windows: `same_field` on syscheck fields not accessible to the correlation engine in the tests of 2026-09-16
100302	3+ modifications within 10 min	CONFIRMED	CONFIRMED	`if_matched_sid` 550
100303	3+ deletions within 10 min	CONFIRMED	CONFIRMED	`if_matched_sid` 553
100304	5+ mixed events (level 14)	CONFIRMED	CONFIRMED	`if_matched_group` `syscheck_file`

Table 12 - Lab validation matrix. Windows endpoints keep creation coverage via 100300 and escalation coverage via 100304

> **Validation scope:** The evidence in this chapter was obtained with the simulation scripts and the real alerts indexed in `wazuh-alerts-*`. Formal controlled-threshold tests of rules 100300 to 100304, with documented scenarios, are outside the scope of this handbook.

6. FIM Alert Investigation

FIM Alert Investigation is a community component built for this research, delivered as the WPTV plugin for the Wazuh Dashboard. It is not part of the Wazuh platform and is not supported by Wazuh.

FIM Alert Investigation is an extension of WAZUH-Process-Tree-Viewer (WPTV), exposed in the **FORENSICS** tab of the Wazuh Dashboard. Its goal is to turn a raw FIM alert into actionable context before the analyst opens the terminal: instead of manually navigating between indices in Discover, the analyst selects the alert, triggers the analysis and receives in seconds a structured verdict with `risk_score`, `risk_level`, MITRE techniques with confidence percentages, IOCs and recommended actions.

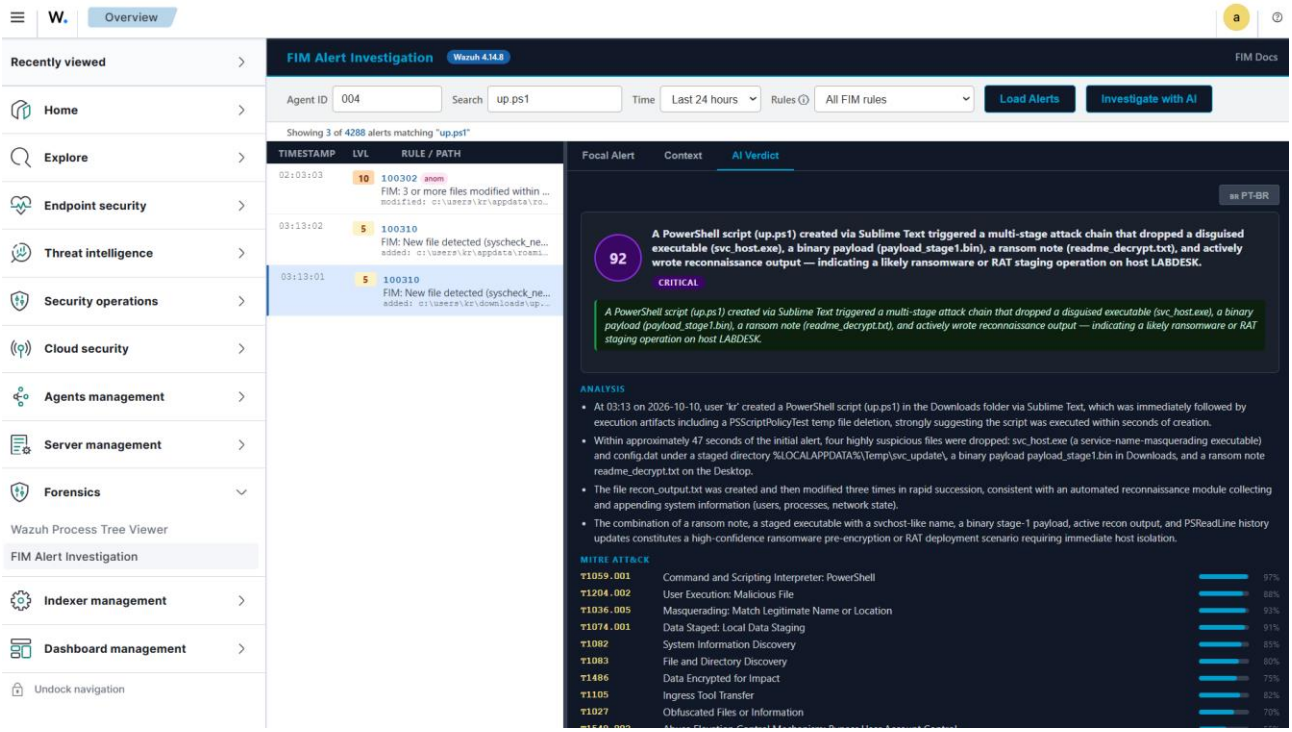


Figure 47 - FIM Alert Investigation interface (Forensics tab)

> **Design principle:** The AI is assisted, not autonomous. The LLM contextualizes and recommends; the analyst decides and executes. No automatic action on the endpoint is triggered by the verdict.

6.1 Architecture

The component runs entirely on the server (WZ-SRV); no additional component is installed on the endpoints. nginx routes `/fim/` and `/api/fim/*` to the same Flask process (`server.py`, port 5000) that already serves WPTV at `/wptv/` and `/api/wptv/*`. There is no separate process, dedicated port or systemd service of its own for FIM Alert Investigation.

nginx route	Destination	Function
`/wptv/`	Flask :5000	WPTV: process tree
`/fim/`	Flask :5000	FIM Alert Investigation: panel
`/api/fim/*`	Flask :5000	FIM API endpoints

Table 13 - nginx routing of WPTV and FIM Alert Investigation

File	Location	Function
`logic_fim.py`	`process_tree_api/`	FIM investigation logic (WPTV extension)
`server.py`	`process_tree_api/`	Flask server with the routes `/api/fim/*`
`fim.html`	`process_tree_api/public/`	FIM Alert Investigation panel

Table 14 - Component files, under /usr/share/wazuh-dashboard/plugins/process_tree_api/

6.2 Investigation Flow

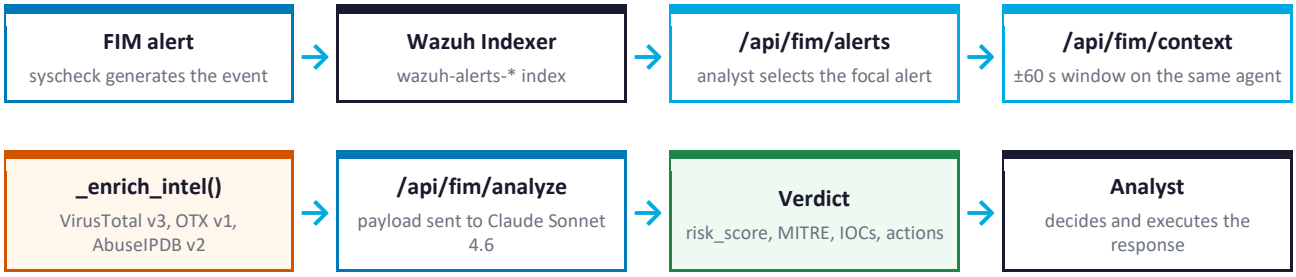


Figure 48 - FIM Alert Investigation flow: from the FIM alert to the structured verdict delivered to the analyst

6.3 ±60-Second Context

The context assembled by `logic_fim.py` aggregates, for the focal alert's agent, four sets of events indexed in the Wazuh Indexer.

Set	Content
`FIM_FILE_RULES`	`{550, 553, 554, 517, 100310}`: file events and the native Syscheck Audit message (517, not a file-event rule); 100310 is the visible creation event (Finding AC-7)
`FIM_REGISTRY_RULES`	`[594, 597, 598, 750, 751, 752]`: Windows Registry events
`FIM_ANOMALY_RULES`	`[100300, 100301, 100302, 100303, 100304]`: anomaly correlations
`PROCESS_EVENTS`	EventID 4688 on Windows; auditd events on Linux

Table 15 - Rule and event sets queried in the context window

6.4 Threat Intelligence Enrichment

The `_enrich_intel()` method queries three external sources for each alert before sending the payload to the LLM.

Source	Query	Return
VirusTotal (API v3)	SHA256 hash at `/files/{hash}`	Detections (`vt_malicious` and `vt_total`), family and permalink
AlienVault OTX (API v1)	SHA256 hash at `/indicators/file/{hash}/general`	Associated pulses; a pulse indicates attribution to known actors or campaigns
AbuseIPDB (API v2)	IPs in `data.audit.addr` in process events captured by auditd	Abuse score (0 to 100), total reports, country and ISP

Table 16 - Enrichment sources of FIM Alert Investigation

IP ENRICHMENT LIMIT: AbuseIPDB is only queried when there is an IP in `data.audit.addr` in process events. IPs present in the content of monitored files are not extracted by the pipeline. In a C2 beacon scenario, the process `connect()` must be captured by auditd (Finding AO-5).

To capture `connect()` on labnix, auditd rules were added and loaded with `augenrules --load`, and confirmed active with `auditctl -l`:

```
# /etc/audit/rules.d/net_connect.rules
-a always,exit -F arch=b64 -S connect -F uid>=1000 -k net_connect_user
-a always,exit -F arch=b32 -S connect -F uid>=1000 -k net_connect_user
-a always,exit -F arch=b64 -S connect -F uid=0 -k net_connect_root
-a always,exit -F arch=b32 -S connect -F uid=0 -k net_connect_root
```

6.5 Wazuh Dashboard Plugin, Logs and Service

- **wptv plugin version 2.1.0:** FIM Alert Investigation ships in the same package as WPTV. Installing the plugin already includes FIM Alert Investigation; the FORENSICS tab is registered by the same `wptv.plugin.js` that registers the WPTV tab.
- **Single log:** all records (`/api/fim/*` requests, LLM analysis, enrichment, `_enrich_intel()` errors) go to `/var/log/wazuh-process-tree/wptv.log`.
- **No service of its own:** the systemd service that manages the process serves both WPTV and FIM Alert Investigation via `server.py`.
- **Model:** Claude Sonnet 4.6, with `max_tokens` of 4096, a backslash-escaping instruction for Windows paths and `JSONDecodeError` handling with logging.

NGINX PREREQUISITE (FINDING AO-4): The POST to `/api/fim/analyze` exceeds the nginx default limit of 1 MB, and nginx responds with HTTP 413 as an HTML page before passing the request to Flask; the frontend fails with `Unexpected token '<'` when parsing the response as JSON. Set `client_max_body_size 10m;` in the `server {}` block of the nginx file `wptv`.

6.6 Lab Validations

Date	Agent	Scenario	Result
2026-10-06	labnix (008)	Focal alert: `PROC_CONTEXT.txt` added in `/home/kr/Downloads/`; 8 process events in the context, with audit key `lpe_quartet_root_netlink_route`	`risk_score` 50, MEDIUM; T1068 with 52% confidence; IOC `/usr/bin/vmtoolsd`; FIM-to-verdict latency of 16 s
2026-10-08	labnix (008)	Alert 100303 (mass deletion), path `/var/tmp/mkinitramfs_prPy58/bin`; MITRE T1485, T1070, T1561	Analysis delivered after the nginx adjustment (Finding AO-4)
2026-10-08	WZ-SRV	`_enrich_intel()` via a test script, with a mock context simulating a C2 beacon and a test IP, with no direct connection from the lab	AbuseIPDB: score 100 and 7691 reports; VirusTotal not_found and OTX 404 (expected for a mock hash)
2026-10-10	labnix (008)	Rule 100310 in the panel after adding `100310` to `FIM_FILE_RULES`; file `/tmp/fim_test_1791649577.txt`, added event, realtime mode	Full description with `[base: rule 554]` and correct `syscheck.*` fields

Date	Agent	Scenario	Result
2026-10-10	LABDESK (004)	Post-exploitation simulation `up.ps1`, in three phases	44 FIM alerts; rule 100302 (level 10); `risk_score` 87, CRITICAL; 11 MITRE techniques; 9 IOCs

Table 17 - FIM Alert Investigation validations

Post-Exploitation Simulation on LABDESK (October 10, 2026)

The script `up.ps1` ran three phases on the LABDESK agent (004): **Phase 1, staging** (creation of `svc_host.exe`, `config.dat`, `update.log`, `payload_stage1.bin` and `readme_decrypt.txt` in AppData\Local\Temp, Downloads and Desktop); **Phase 2, reconnaissance** (PowerShell to cmd.exe, with `whoami`, `systeminfo`, `ipconfig`, `netstat`, `net`, `tasklist`, `arp` and `route`, 11 commands in total, with output to `recon_output.txt`); **Phase 3, lateral copy** (`svc_host.exe` copied to `C:\Windows\Temp\` and Desktop). The correlation of FIM with the process tree (EventID 4688) was confirmed in the panel.

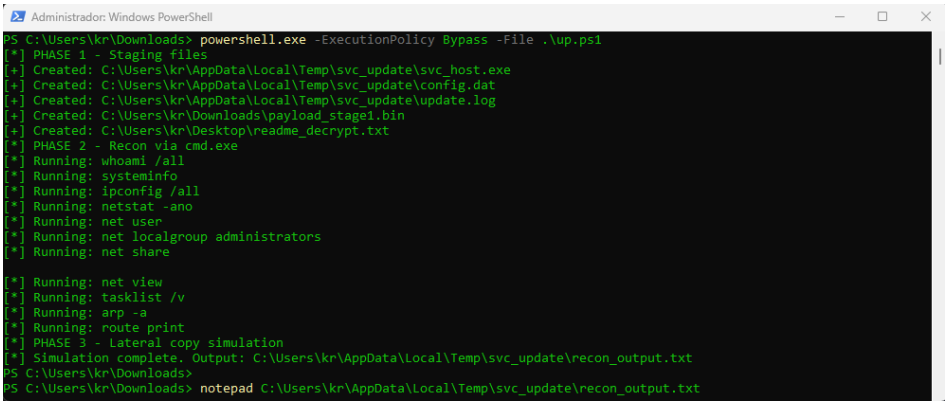


Figure 49 - Execution of the up.ps1 simulation script on LABDESK: staging, reconnaissance and lateral copy phases

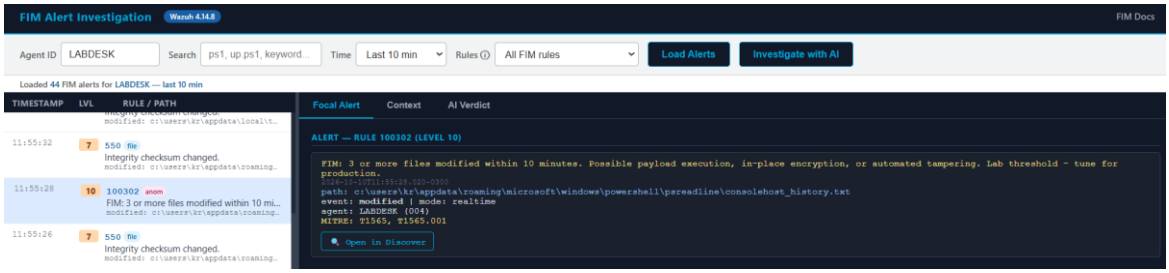


Figure 50 - FIM Alert Investigation panel, Focal Alert tab: rule 100302 (level 10), 44 alerts loaded for LABDESK in the last 10 min; MITRE T1565 and T1565.001

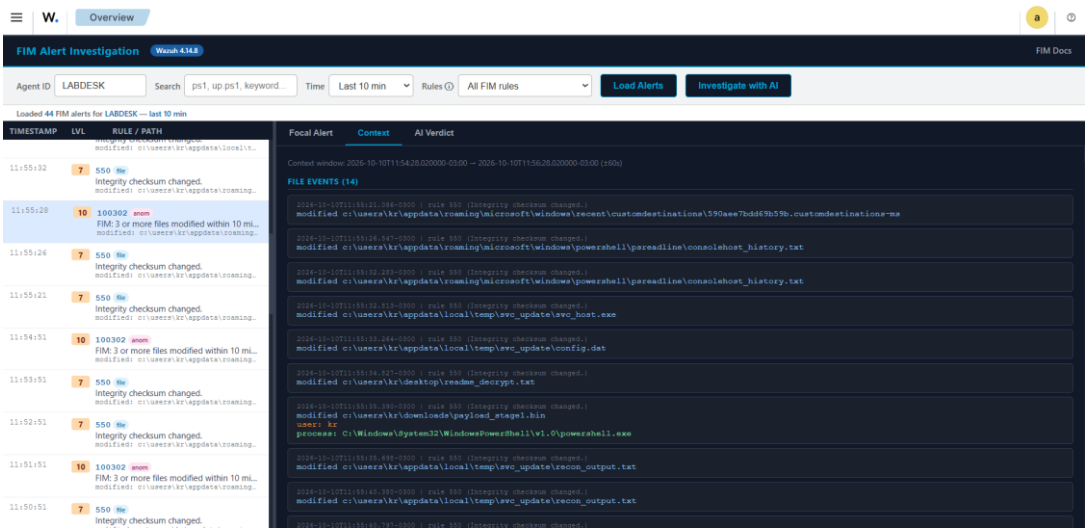


Figure 51 - Context tab: 14 file events in the ±60 s window of the selected 100302 alert

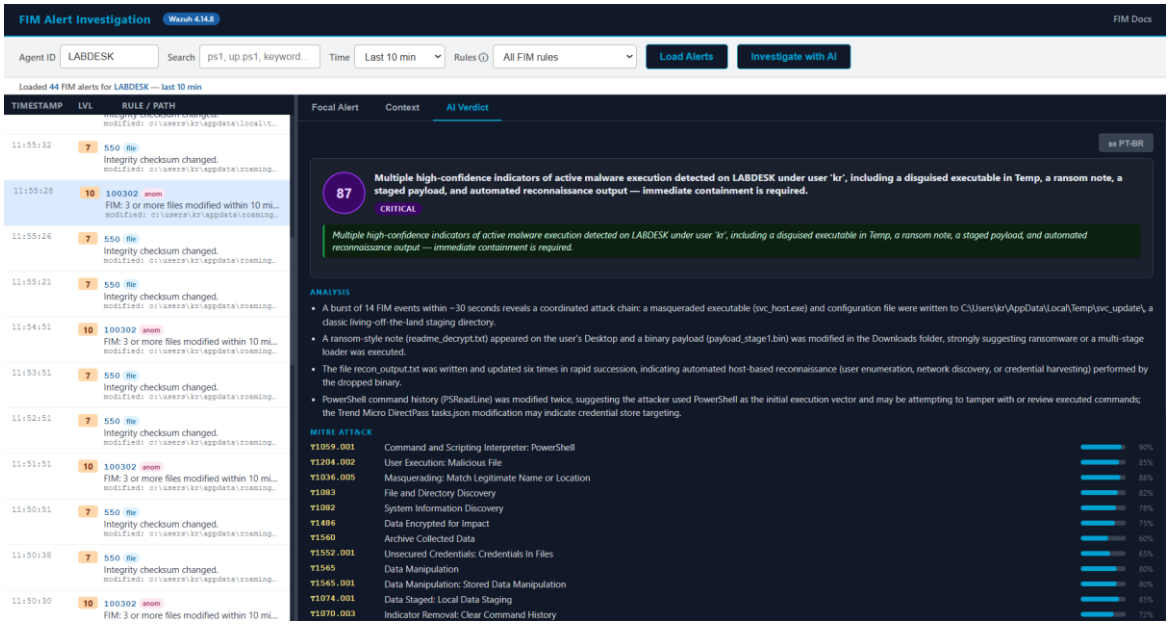


Figure 52 - AI Verdict tab: risk_score 87, CRITICAL level, with MITRE techniques and confidence percentages

MITRE technique	Name	Confidence
T1059.001	Command and Scripting Interpreter: PowerShell	90%
T1036.005	Masquerading: Match Legitimate Name or Location	88%
T1204.002	User Execution: Malicious File	85%
T1083	File and Directory Discovery	82%
T1565 and T1565.001	Data Manipulation and Stored Data Manipulation	80%
T1082	System Information Discovery	78%
T1486	Data Encrypted for Impact	75%
T1074.001	Data Staged: Local Data Staging	75%
T1070.003	Indicator Removal: Clear Command History	72%

MITRE technique	Name	Confidence
T1552.001	Unsecured Credentials: Credentials In Files	65%
T1560	Archive Collected Data	60%

Table 18 - MITRE ATT&CK techniques in the up.ps1 simulation verdict (risk_score 87, CRITICAL)

The verdict identified 9 IOCs: 2 hashes (SHA256 and MD5 of the PSReadLine `ConsoleHost_History.txt`) and 7 suspicious paths, among them `svc_host.exe`, `recon_output.txt`, `readme_decrypt.txt`, `payload_stage1.bin`, `config.dat` and `tasks.json` from a password manager. The three recommended actions were: immediate network isolation of the host, forensic collection with sandbox analysis, and an audit of authentication events with hardening through AppLocker or WDAC.



Figure 53 - AI Verdict tab: indicators of compromise (hashes and paths) and recommended actions

> **Reading the verdict:** The `risk_score` of 87 refers to simulated lab activity. In production, the LLM verdict is a triage input that speeds up the analyst's decision; it does not replace alert validation or the containment decision.

7. Complementary Rules from the Default Ruleset

The `0015-ossec_rules.xml` file that ships with Wazuh 4.14.x contains rules that are not FIM rules as such, but determine the effectiveness of integrity monitoring in production. Knowing them separates a reactive FIM deployment from a resilient one.

Rule ID	Level	Description	Relevance to FIM	MITRE
504	3	Wazuh agent disconnected	Start of a possible blind window	T1562.001
505	3	Wazuh agent removed	Agent removed: monitoring ended	T1562.001
506	3	Wazuh agent stopped	Service stopped: FIM ceases completely and silently	T1562.001
560	7	FIM real-time queue is full. Some real-time events may be lost.	Kernel inotify queue full: realtime events discarded	-
594 to 598, 750 to 752	variable	Registry key and value events	Windows Registry FIM (chapter 3)	T1112 and related

Table 19 - Native rules that condition FIM reliability

7.1 Agent Lifecycle and Blind Windows (504, 505, 506)

Rules 504, 505 and 506 cover agent disconnection, removal and stop, all mapped to MITRE T1562.001 (Impair Defenses: Disable or Modify Tools). When the agent stops, all integrity monitoring on that endpoint ceases immediately and silently: no file event is generated, no alert reaches the server, and the endpoint merely changes its status icon to disconnected.

```
<rule id="504" level="3">
  <if_sid>500</if_sid>
  <match>Agent disconnected</match>
  <description>Wazuh agent disconnected.</description>
  <mitre><id>T1562.001</id></mitre>
</rule>
<rule id="505" level="3">
  <if_sid>500</if_sid>
  <match>Agent removed</match>
  <description>Wazuh agent removed.</description>
  <mitre><id>T1562.001</id></mitre>
</rule>
<rule id="506" level="3">
  <if_sid>500</if_sid>
  <match>Agent stopped</match>
  <description>Wazuh agent stopped.</description>
  <mitre><id>T1562.001</id></mitre>
</rule>
```

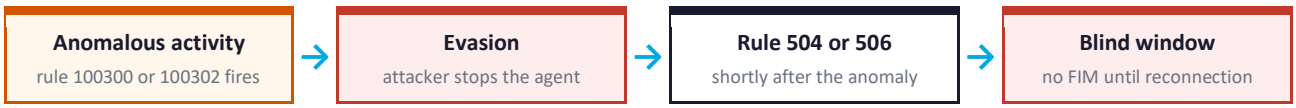


Figure 54 - High-fidelity sequence: anomalous FIM followed by agent disconnection or stop

The threat pattern is the sequence 100300 or 100302 followed by 504 or 506 within a short interval: an attacker who notices FIM is active may shut down the agent before proceeding. Correlating anomalous FIM immediately preceding a disconnection event is a high-fidelity behavioral indicator, worth implementing as a custom rule in high-criticality environments.

RECONNECTION IS NOT RETROACTIVE: When the agent reconnects after an offline period, `syscheckd` runs a new baseline scan and synchronizes the current state. Files created, modified and deleted during the offline period do not generate retroactive events. Any gap in the FIM history preceded by a 504 or 506 event must be investigated as a possible window of unmonitored activity, and not dismissed as an absence of activity.

7.2 Real-Time inotify Queue Overflow (560)

Rule 560 alerts when the kernel's internal inotify queue has reached maximum capacity and real-time events are being discarded. It is the native alert for the watch limit problem of chapter 4. Level 7, syscheck group, with an unambiguous description: "Some real-time events may be lost".

```
<rule id="560" level="7">
  <if_sid>500</if_sid>
  <match>^ossec: Real-time inotify kernel queue is full.</match>
  <description>FIM real-time queue is full. Some real-time events may be lost.</description>
  <group>syscheck,agentless,pci_dss_11.5,...</group>
</rule>
```

The presence of 560 on a Linux endpoint is a direct sign that `fs.inotify.max_user_watches` needs to be increased or that the number of directories with realtime or whodata needs to be reviewed. In environments under PCI DSS or HIPAA, a 560 alert is technically a violation of the integrity control: the tool explicitly reports that it cannot guarantee real-time coverage.

What 560 does not cover: Rule 560 depends on the kernel queue overflowing. It does not indicate whether realtime is delivering events to the manager. To confirm delivery, query `archives.json` or the `wazuh-archives-*` index and check that `syscheck.mode` equals `realtime` (chapter 4).

7.3 Windows Registry

Rules 594, 597, 598, 750, 751 and 752 cover Registry FIM and are detailed in section 3.7. The relevant operational point here is the correlation with the file system: persistence through a Run key generates a 554 alert on the dropped file and a 752 on the created value, correlatable by agent and time window.

7.4 Active Response

Active Response is the Wazuh capability to run automatic actions on endpoints in response to alerts. The detailed configuration and use of Active Response is outside the scope of this handbook: including it would increase the page count and could divert the focus of the study, which is dedicated to FIM. For those who wish to use it, the reference is the official Wazuh documentation, cited at the end of this section.

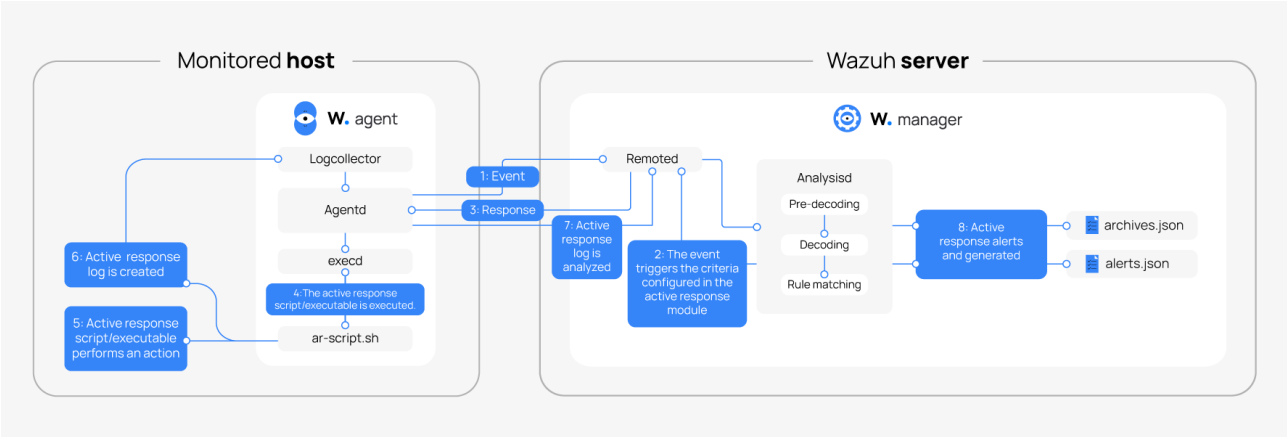


Figure 55 - Active Response workflow between the Wazuh agent on a monitored host and the Wazuh manager on the Wazuh server. Source: Wazuh documentation, Active Response (https://documentation.wazuh.com/current/user-manual/capabilities/active-response/)

> **Types of active response:** "An active response can either be: Stateless, Stateful. Stateless active responses are one-time actions without an event definition to revert or stop them. Stateful responses revert or stop their actions after a period of time." (Wazuh documentation, Active Response)

In short: stateless responses are one-time actions, with no event definition to revert or stop them; stateful responses revert or stop their actions after a period of time.

Official documentation: https://documentation.wazuh.com/current/user-manual/capabilities/active-response/

8. FIM from the Adversary's Perspective

This chapter approaches FIM from an advanced defensive perspective: the techniques an attacker with access to the system can use to interfere with integrity monitoring and the countermeasures Wazuh offers to detect that interference. Understanding how detection can fail is a necessary condition for building a resilient monitoring strategy.

8.1 Evasion Techniques and Countermeasures

Technique	Platform	How it interferes	Countermeasure
Removal of audit rules (`auditctl -D`)	Linux	Root removes all loaded rules, including Wazuh's; whodata stops receiving events from that moment on, which creates a blind window	The `CONFIG_CHANGE` event in Audit generates rule 80705; filtering by that ID and cross-referencing `audit.exe` with the responsible process distinguishes legitimate Wazuh auto-configuration from another process
Audit policy reversion (`auditpol /clear` or PowerShell)	Windows	Reverts File System and Handle Manipulation	EventID 4719 (System audit policy was changed) with the GUID of the two subcategories and without the Wazuh process as originator indicates interference; `windows_audit_interval` detects the change and switches whodata directories to realtime
Agent stop or removal	Both	Ceases all monitoring of the endpoint	Rules 504, 505 and 506 (T1562.001) and correlation with anomalous FIM immediately beforehand
Timestomping	Both	Sets `mtime` to the past to hide when the file was created or modified	`syscheck.mtime_after` far earlier than the alert timestamp; mapped to T1070.006
Atomic replacement via `rename()`	Both	A malicious file created outside monitoring atomically replaces a monitored file; FIM sees a modification, not a creation	`report_changes` on the affected directory captures the before and after diff, visible in the `diff` field of the alert

Table 20 - Evasion techniques against FIM and countermeasures available in Wazuh

8.2 Timestomping

On Linux, `mtime` can be changed directly with `touch -t` or the `utimes` system call. What cannot be changed from userland is `ctime`, the inode metadata change timestamp, which the kernel updates automatically on every metadata change, including that of `mtime` itself. This creates a detectable signature: a FIM event with `mtime_after` in the past while the alert arrives in the present indicates a file with a manipulated `mtime`. Wazuh captures `mtime_after` in the event, and comparing it with the alert timestamp in Discover reveals timestomping, a high-fidelity indicator. The mapping to MITRE ATT&CK T1070.006 is this handbook's (the native rule 550 maps only T1565.001), and files copied with preserved timestamps (cp -p, tar, package installs) also show an old mtime_after, so confirm with the process attribution.

A DQL query in Discover exposes the anomaly:

```
rule.id:550 AND syscheck.mode:whodata
```

After filtering, sort the `syscheck.mtime_after` column in ascending order: the oldest dates stay at the top and are the candidates. Confirmation is the comparison with the alert timestamp; a difference of days or weeks on a file in a critical directory such as `/usr/bin` or `System32` deserves investigation. For attribution, the

`syscheck.audit.process.name` field (Linux) or `audit.process.name` (Windows) indicates the responsible executable; `touch` on Linux or `attrib` and PowerShell on Windows as authors of a system binary modification are a strong additional indicator.

8.3 Ransomware Event Pattern

With realtime or whodata on critical directories, FIM produces a recognizable pattern during a ransomware infection: encryption of existing files generates an avalanche of 550 events in a short interval, on the same agent and often in the same directory, frequently with an extension change in `syscheck.path` indicating the rename to the ransomware extension.



Figure 56 - Ransomware signature in FIM: burst of modifications, renames and escalation through correlation

The pattern in Figure 56 (burst of modifications, renames and escalation through correlation) is covered by the validated rules 100302 (3 or more 550 events in 10 minutes, level 10) and 100304 (escalation on mixed activity, level 14, which maps T1486 - Data Encrypted for Impact). The definitions, identical to those in section 5.3, are reproduced below.

```
<rule id="100302" level="10" frequency="3" timeframe="600" ignore="30">
  <if_matched_sid>550</if_matched_sid>
  <description>FIM: 3 or more files modified within 10 minutes. Possible payload execution,
in-place encryption, or automated tampering. Lab threshold - tune for
production.</description>
  <group>suspicious_modification,</group>
  <mitre><id>T1565</id><id>T1565.001</id></mitre>
</rule>

<rule id="100304" level="14" frequency="5" timeframe="600" ignore="30">
  <if_matched_group>syscheck_file</if_matched_group>
  <description>FIM: High volume mixed file activity - 5 or more create, modify, and delete
events within 10 minutes. Combined pattern suggesting active malware execution. Lab
threshold - tune for production.</description>
  <group>suspicious_combined,fim_escalation,</group>
  <mitre><id>T1485</id><id>T1486</id><id>T1565</id></mitre>
</rule>
```

THRESHOLD CALIBRATION: The `frequency="3"` and `timeframe="600"` values of rules 100302 and 100304 are lab values, as noted in chapter 5. In environments with high legitimate file activity, such as build servers and backup systems, tune them to the profile of each agent group before deployment. Without whodata, the rules detect the behavior but do not attribute the responsible process; with whodata, the alert carries the responsible process (section 8.2).

8.4 Cataloged Blind Windows

The table consolidates the situations in which FIM stops covering an endpoint without this being evident, with the origin documented in this handbook.

Blind window	Cause	Observable signal	Reference
Agent stopped or disconnected	Evasion or failure	Rules 504, 505, 506; disconnected status	7.1

Blind window	Cause	Observable signal	Reference
Activity during offline period	Delta synchronization	Zero delta on reconnection; gap in the history	2.7 and 7.1
Audit rules removed	`auditctl -D` or persistence failure	Rule 80705; empty `auditctl -l`	4.3 and 4.4
Audit policy reverted	GPO or `auditpol /clear`	EventID 4719	3.4 and 3.5
SACL not recovered after manager restart	whodata not restored after the reconnection; `windows_audit_interval` does not reapply the SACL	No EventID 4719 after reconnection; no whodata events	AO-7
Whodata with silent fallback to realtime	Orphaned socket or inaccessible auditd	Events without user attribution	4.6 and 4.7
inotify queue full	`max_user_watches` insufficient	Rule 560	4.8 and 7.2
Creation alerts hidden by the proxy rule	Child rule 100310 at level 0 suppresses the alert of 554	`archives.json` with `syscheck.mode` realtime, `added` event and null `rule`; absence of 554 in `wazuh-alerts-*`	AC-7
Watches on individual files	Path with `/*` in `<directories>`	6016 per file in `ossec.log`	AO-8

Table 21 - FIM blind windows and how to recognize them

8.5 Sysmon as a Complementary Layer to FIM

FIM reports that a file changed, which attributes changed and, with whodata, which process and which user performed the operation. Sysmon observes the same activity from the operating system side and records events that FIM does not carry, such as the change of a file's creation date (EID 2), the creation or overwriting of files (EID 11) and detected deletion (EID 26). For this reason Sysmon is treated in this handbook as a complementary layer, which adds to FIM and does not replace it.

For this layer, the Unified Sysmon Configs repository, https://github.com/mym0us3r/Unified-Sysmon-Configs, was used as the basis of the native Sysmon configuration (schema 4.91) on the lab's Windows 11 endpoint (LABDESK).

From that repository, tests were run with a new research-oriented Wazuh ruleset, with five rules in the 100500 to 100599 range (100521, 100540, 100541, 100554 and 100555). Each Sysmon rule matches a real FIM alert for the same file operation: creation, overwrite, deletion, executable copy and date change. The set was made available on GitHub so that the community can test, modify and adapt the rules to the reality of the environment where they are implemented.

> **RESEARCH RULESET:** The Sysmon rules are a starting point for testing and are not part of this handbook's validated set of six FIM rules. The monitored path (`D:\File_Server_Test_Folder`), the levels and the interpreter list must be adjusted to the environment before any production use.

Evidence in Wazuh Discover

Figures 57 to 68 gather the real lab captures, in the order in which the operations were run on LABDESK on October 12, 2026. Figure 57 shows the application of the Sysmon configuration. The others are Wazuh Discover queries, each identified by the rule.id of the alert: rules 100521, 100540, 100541, 100554 and 100555 are the Sysmon rules of this research ruleset, and rules 550, 553, 100300, 100302, 100303 and 100310 are the

handbook's FIM rules. Figures 59, 61 and 62 document the operations in which FIM is the only source: the overwrite with Set-Content, the append and the rename did not generate a Sysmon alert.



Figure 57 - XML validation and application of the native Sysmon configuration with sysmon -c on LABDESK (schema 4.91)

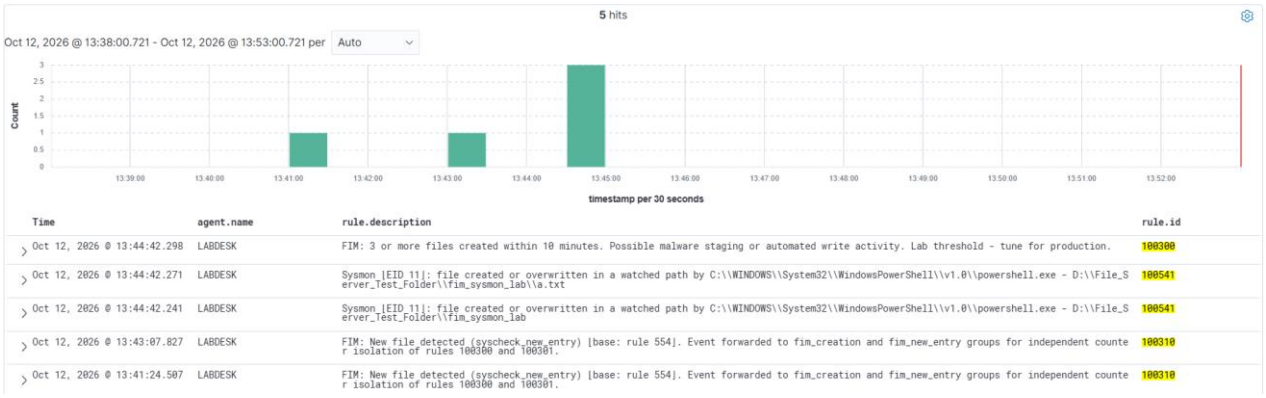


Figure 58 - Creation of a.txt on LABDESK: rule 100541 (Sysmon, EID 11, 13:44:42.241 and 13:44:42.271) followed by 100300 (FIM, 13:44:42.298)



Figure 59 - Overwrite with Set-Content: alert 550 (FIM) at 14:14:09.810, with no associated Sysmon alert (interval from 14:13 to 14:21)



Figure 60 - Overwrite with cmd.exe redirection: rules 100555 (Sysmon, EID 26, 14:18:57.882) and 100541 (Sysmon, EID 11, 14:18:57.966) with the cmd.exe image

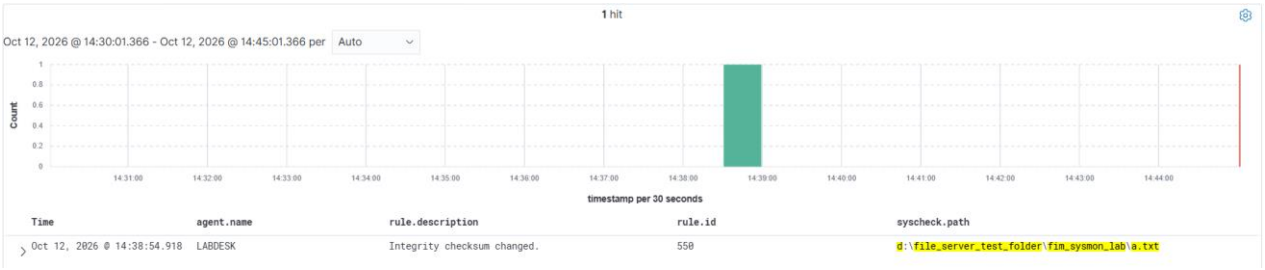


Figure 61 - Append a line to a.txt: combined query by path (FIM or Sysmon) returns only the 550 alert (FIM) of 14:38:54.918

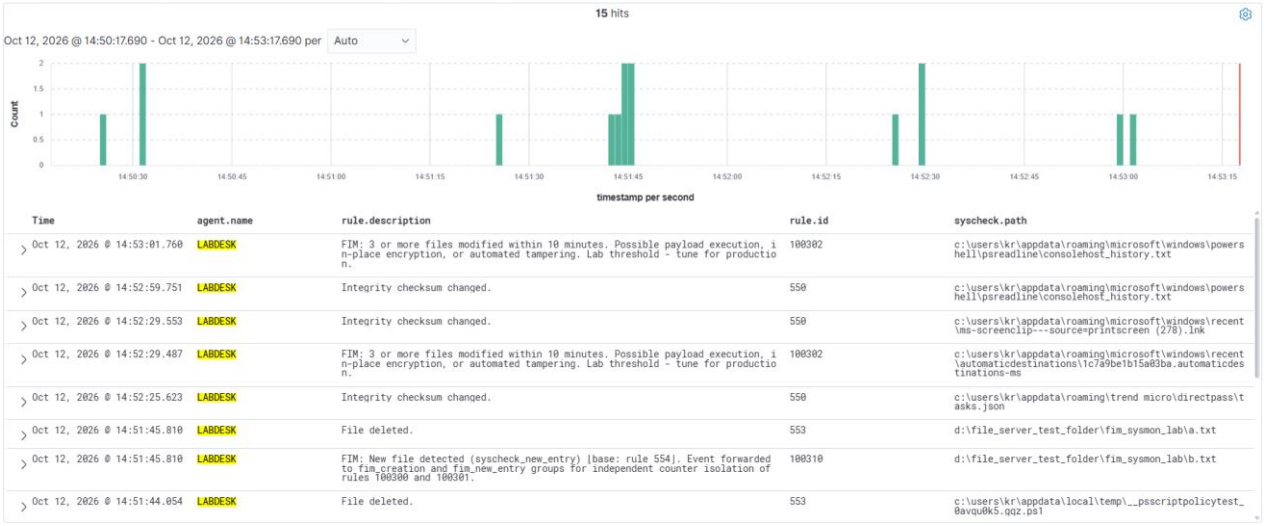


Figure 62 - Rename files: FIM alerts 553 (a.txt) and 100310 (b.txt) at 14:51:45.810, with no associated Sysmon alert



Figure 63 - Deletion of b.txt: alert 553 (FIM) and alert 100555 (Sysmon, EID 26) on the same path



Figure 64 - Copy of the executable tool_ps.exe by PowerShell: alert 100540 (Sysmon, EID 11)

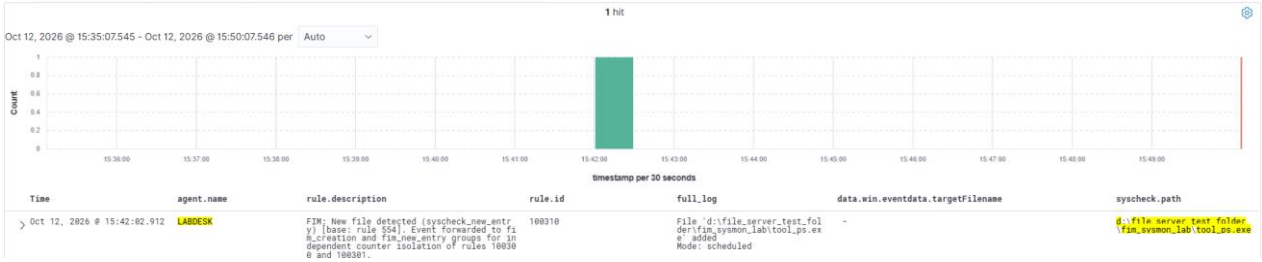


Figure 65 - Same executable in FIM: alert 100310 (proxy of 554, scheduled mode) with the file's MD5 and SHA-256 hashes



Figure 66 - Date change of a.txt (timestomp): alert 100521 (Sysmon, EID 2)

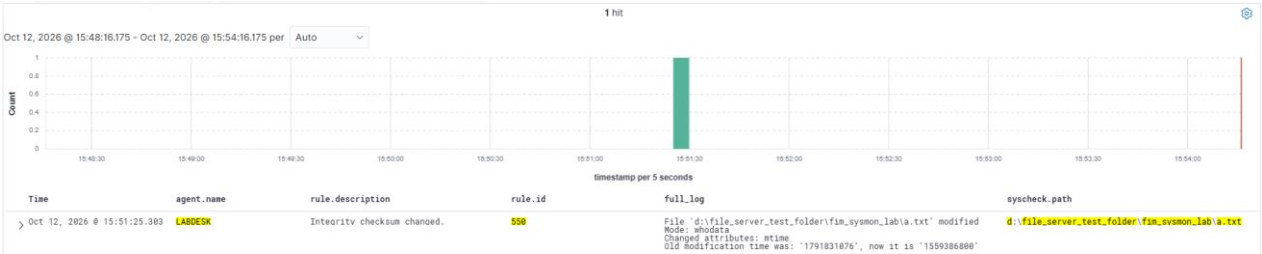


Figure 67 - Same operation in FIM: alert 550 (whodata mode) with the mtime attribute changed



Time	agent.name	rule.description	rule.id	full_log	syscheck.path
Oct 12, 2026 @ 15:56:26.953	LABDESK	Integrity checksum changed.	550	File 'c:\users\kr\appdata\roaming\trend micro\directpass\tasks.json' modified Mode: realtime Changed attributes: mtime Old modification time was: '1791831327', now it is '1791831387'	c:\users\kr\appdata\roaming\trend micro\directpass\tasks.json
Oct 12, 2026 @ 15:56:10.793	LABDESK	FIM: 3 or more files deleted within 10 minutes. Possible evidence destruction, anti-forensics, or post-execution malware cleanup. Lab threshold - tune for production.	100303	File 'd:\file_server_test_folder\fim_sysmon_lab\tool_ps.exe' deleted Mode: whodata	d:\file_server_test_folder\fim_sysmon_lab\tool_ps.exe
Oct 12, 2026 @ 15:56:09.893	LABDESK	Sysmon [EID 26]: executable or script deleted by interpreter C:\\WINDOWS\\System32\\WindowsPowerShell\\v1.0\\powershell.exe - D:\\File_Server_Test_Folder\\fim_sysmon_lab\\tool_ps.exe	100554	-	-
Oct 12, 2026 @ 15:56:09.864	LABDESK	Sysmon [EID 26]: file deleted in a watched path by C:\\WINDOWS\\System32\\WindowsPowerShell\\v1.0\\powershell.exe - D:\\File_Server_Test_Folder\\fim_sysmon_lab\\a.txt	100555	-	-
Oct 12, 2026 @ 15:56:09.851	LABDESK	File deleted.	553	File 'd:\file_server_test_folder\fim_sysmon_lab\a.txt' deleted Mode: whodata	d:\file_server_test_folder\fim_sysmon_lab\a.txt
Oct 12, 2026 @ 15:56:08.873	LABDESK	FIM: 3 or more files modified within 10 minutes. Possible payload execution, in-place encryption, or automated tampering. Lab threshold - tune for production.	100302	File 'c:\users\kr\appdata\roaming\microsoft\windows\powershell\psreadline\consolehost_history.txt' modified Mode: realtime Changed attributes: size,mtime,md5,sha1,sha256 Size changed from '146306' to '146388' Old modification time was: '1791831367', now it is '1791831369' Old md5sum was: 'c7b98ab91a6b594ab0a3ce5b2a9024	c:\users\kr\appdata\roaming\microsoft\windows\powershell\psreadline\consolehost_history.txt

Figure 68 - Cleanup of the test files: 553 and 100555 (a.txt), 100554 (Sysmon, EID 26) and 100303 (FIM) for tool_ps.exe

9. Compliance and MITRE ATT&CK

One of the most frequent questions in the Wazuh community is how to map the FIM configuration to the requirements of each compliance framework. Wazuh automatically tags each alert with the applicable identifiers, visible in the fields `rule.pci_dss`, `rule.hipaa`, `rule.nist_800_53`, `rule.gdpr` and `rule.tsc`. This section details what each requirement demands and how FIM meets it.

Framework	Requirement	FIM coverage	Alert field
PCI DSS 4.0	11.5.2: mechanism for detecting changes to critical files, with comparison at least weekly	Scheduled scan with `frequency` of up to 604800 s meets the minimum; realtime or whodata exceeds the requirement	`rule.pci_dss (11.5 in native rules)`
PCI DSS 4.0	10.3.3: audit logs securely copied to a centralized server	Agents send to the server, which sends to the indexer	`rule.pci_dss`
NIST 800-53 Rev 5	SI-7: software, firmware and information integrity verification	MD5, SHA-1 and SHA-256 checksums (SI-7(6)); scan and real time (SI-7(1)); automatic alerts (SI-7(2))	`rule.nist_800_53` (SI.7)
HIPAA	§164.312(c)(1) and (c)(2): protection and authentication of ePHI against improper alteration or destruction	FIM with whodata detects modification and records user and process	`rule.hipaa` (164.312.c.1 and c.2)
GDPR	Art. 5(1)(f): adequate security, including protection against unauthorized processing and against loss or destruction	FIM with whodata on directories containing personal data	`rule.gdpr` (II_5.1.f)
SOC 2 (TSC)	CC6.1, CC6.8, CC7.2 and CC7.3	Logical access, malicious software prevention, anomaly monitoring and incident evaluation	`rule.tsc`

Table 22 - Mapping of compliance requirements to Wazuh FIM

9.1 PCI DSS 4.0

The most direct requirement is 11.5.2: a change-detection mechanism, such as file integrity monitoring, must alert personnel to unauthorized modifications of critical system, configuration or content files, scoped to the CDE systems, and perform critical file comparisons at least weekly. The requirement does not specify a log format, but expects integrity events to be retained and searchable, which the `wazuh-alerts-*` index with configured retention satisfies. Requirement 10.3.3, secure copying of audit logs to a centralized server, is structurally met by the agent, server and indexer architecture. Note that the native Wazuh rules tag alerts with the earlier numbering (for example pci_dss_11.5 on rules 550, 553 and 554), so the rule.pci_dss field shows 11.5 and not 11.5.2.

9.2 NIST 800-53 Rev 5

The central control is SI-7, which requires integrity verification tools to detect unauthorized modifications. Wazuh FIM implements SI-7 with MD5, SHA-1 and SHA-256 checksums, mentioned as cryptographic protection in enhancement SI-7(6). SI-7(1) requires checks by automated tools, met by the scan and by real time; SI-7(2) requires automatic notification of violations, met by the alerting mechanism; SI-7(7) requires integration with the incident response process. The `rule.nist_800_53` field uses the identifier SI.7 in all alerts of rules 550, 553 and 554.

9.3 HIPAA

The controls are in §164.312, Technical Safeguards. §164.312(c)(1) requires policies to protect ePHI from improper alteration or destruction; §164.312(c)(2) requires a mechanism to authenticate ePHI, that is, to confirm that it has not been altered or destroyed in an unauthorized manner. For healthcare environments, the recommended configuration is `whodata` with `report_changes` on clinical data directories and on the configuration files of electronic health record systems, so that any unauthorized modification produces an alert with full user attribution and content diff.

9.4 GDPR and TSC

The `rule.gdpr` field uses the identifier II_5.1.f, corresponding to Article 5(1)(f), which requires personal data to be processed with adequate security, including protection against unauthorized or unlawful processing and against accidental loss, destruction or damage. FIM with whodata on directories containing personal data implements this principle by detecting and recording any unauthorized access or modification. Wazuh also tags alerts with the Trust Services Criteria CC6.1, CC6.8, CC7.2 and CC7.3.

> **Audit tip:** The Wazuh Dashboard has prebuilt compliance dashboards under Endpoint Security, which filter and group alerts by framework. They display FIM events already mapped to each control, which reduces the effort of preparing evidence for external auditors.

9.5 MITRE ATT&CK Mapping

The correlation rules were mapped to the relevant MITRE ATT&CK techniques, which allows incident response teams to correlate FIM alerts with the adversary's attack chain.

Technique	ID	Rules	Detection scenario
Data Manipulation	T1565	100300, 100302, 100304	Mass creation and modification of files
Data Manipulation: Stored Data Manipulation	T1565.001	100300, 100302, 594, 750	Local data integrity manipulation
Data Staged	T1074	100300	File preparation for staging or exfiltration
Ingress Tool Transfer	T1105	100301	Binary propagation via identical hash
Lateral Tool Transfer	T1570	100301	Tool replication between directories
Data Destruction	T1485	100303, 100304, 597, 751	Mass destruction of files and evidence
Indicator Removal	T1070	100303	Trace removal and anti-forensics
Disk Wipe	T1561	100303	Disk cleanup after malware execution
Data Encrypted for Impact	T1486	100304	Mixed activity suggesting ransomware
Modify Registry	T1112	594, 597, 598, 750, 751, 752	Persistence and configuration via the Windows Registry
Impair Defenses: Disable or Modify Tools	T1562.001	504, 505, 506	Agent stop, removal or disconnection

Technique	ID	Rules	Detection scenario
Indicator Removal: Timestomp	T1070.006	550 (hunting by `mtime_after`)	Timestamp manipulation (section 8.2)

Table 23 - Mapping of FIM rules to MITRE ATT&CK techniques

10. Validated Technical Findings

The findings below were identified, investigated and handled throughout the development of the pipeline, all with evidence collected in the lab environment, without assumptions and without simulated output; test inputs, when used, such as the mock context in Table 17, are identified as such. Each finding presents the observed behavior, the impact and the mitigation. The IDs in this handbook unify two sources that use their own numbering: **AC** (correlation findings in analysisd, "LAB FINDINGS" in the rules file) and **AO** (operational findings from the pipeline handoff). Findings AC-1 to AC-7 and AO-1 to AO-8 are observations from the lab on 4.14.x, not behaviors stated in the official Wazuh documentation.

ID	Finding	Type	Original numbering	Status
AC-1	same_field must use `md5_after`	Correlation	LAB FINDING [1]	RESOLVED
AC-2	`if_matched_sid` with multiple SIDs	Correlation	LAB FINDING [2]	RESOLVED
AC-3	Counter interference between 100300 and 100301	Correlation	LAB FINDING [3]	RESOLVED
AC-4	`wazuh-logtest` does not process syscheck	Validation	LAB FINDING [4]	EXPECTED
AC-5	Interference confirmed on Linux	Correlation	LAB FINDING [5]	RESOLVED
AC-6	Child-parent interference on 100300	Correlation	LAB FINDING [6]	RESOLVED
AC-7	Level of 100310 and suppression of 554	Correlation	LAB FINDING [7]	RESOLVED
AO-1	`field name="file"` and not `syscheck.path`	Rules	Handoff, finding 1	RESOLVED
AO-2	Saturation by noise sources	Endpoint	Handoff, finding 2	RESOLVED
AO-3	`auditpol` on a Windows workgroup	Endpoint	Handoff, finding 3	RESOLVED
AO-4	`client_max_body_size` in nginx	Integration	Handoff, finding 4	RESOLVED
AO-5	AbuseIPDB via auditd `connect`	Integration	Handoff, finding 5	RESOLVED
AO-6	VirusTotal and custom payloads	Integration	Handoff, finding 6	EXPECTED
AO-7	SACL not recovered after manager restart	Windows	Handoff, finding 7	MITIGATED
AO-8	Path with `/*` expands the glob	Endpoint	Handoff, finding 8	RESOLVED

Table 24 - Index of the 15 technical findings

10.1 Correlation in analysisd (AC)

AC-1 Correct mapping of the same_field field

RESOLVED Correlation (analysisd)

Observation	In FIM correlation rules, `same_field` must reference the field name decoded by the analysisd decoder. For FIM events the correct field is `md5_after`, not `md5`.
Evidence	Comparison between the alert JSON in `wazuh-alerts-*` and the output of the internal analysisd decoder. In the lab (4.14.x), same_field with md5 does not correlate.
Impact	With `md5`, rule 100301 does not correlate: the field is not found in the event and the accumulator does not advance.

Mitigation	`<same_field>md5_after</same_field>` in rule 100301.
Origin	LAB FINDING [1] of `fim_anomaly_rules.xml`

AC-2 if_matched_sid with multiple values is not reliable

RESOLVED Correlation (analysisd)

Observation	The syntax of comma-separated SIDs in `if_matched_sid` (for example, `550,553,554`) is not reliably supported in temporal correlation rules in 4.14.x.
Mitigation	Use `if_matched_group` with a shared group. Rule 100304 uses `syscheck_file`, the native group of rules 550, 553 and 554, confirmed in `0015-ossec_rules.xml`.
Origin	LAB FINDING [2]

AC-3 Counter interference between rules 100300 and 100301

RESOLVED Correlation (analysisd)

Observation	With `if_matched_sid` 554 on both, analysisd shares the event accumulator: the firing of 100300 on the third event consumes the counter state and prevents 100301 from triggering correctly.
Mitigation	Auxiliary rule 100310 (child of 554) re-forwards each 554 event to the groups `fim_creation` (100300) and `fim_new_entry` (100301), creating two independent accumulators.
Origin	LAB FINDING [3], initially identified on LABDESK (Windows)

AC-4 wazuh-logtest does not process syscheck events

EXPECTED Validation tool

Observation	`wazuh-logtest` does not process syscheck events in phases 2 and 3. The `syscheck_new_entry`, `syscheck_integrity_changed` and `syscheck_deleted` decoders are internal to the analysisd pipeline and unavailable in logtest.
Impact	FIM rules can only be validated through the real pipeline, with an agent generating events. Each test cycle requires deploying the rules, restarting `wazuh-manager` and generating real events on the endpoints.
Origin	LAB FINDING [4]

AC-5 Counter interference confirmed on Linux

RESOLVED Correlation (analysisd)

Observation	AC-3 was identified on Windows (LABDESK). Tests on labnix (October 9, 2026) confirmed the same behavior on Linux when 100300 and 100301 share `if_matched_sid` 554. The interference is a characteristic of analysisd on the server, independent of the agent's operating system.
Mitigation	The 100310 proxy pattern with separate groups is effective and validated on both platforms.
Origin	LAB FINDING [5]

AC-6 Child-parent interference on rule 100300 RESOLVED Correlation (analysisd)	
Evidence	100303 (`if_matched_sid` 553, no child rule on 553) fires in every test; 100300 (`if_matched_sid` 554, with the child 100310 on 554) did not fire. The child rule event, inserted between consecutive 554 events, disorganized the accumulator.
Mitigation	100310 emits to `fim_creation`, and 100300 moved from `if_matched_sid` 554 to `if_matched_group` `fim_creation`, completely isolating its accumulator.
Origin	LAB FINDING [6]

AC-7 Level of rule 100310 and suppression of rule 554 RESOLVED Correlation (analysisd)	
Observation	With the child rule 100310 (`if_sid` 554) at a level greater than zero, analysisd suppresses the alert of the parent rule 554: only 100310 reaches `wazuh-alerts-*`. The level 0 attempt suppressed the alert entirely, leaving the event only in `wazuh-archives-*`, with no `rule` field.
Evidence	Scan of `alerts.json`: more than 40 entries with `rule.id=100310` and zero with `rule.id=554`. Validated on October 10, 2026: LABDESK at 02:14:29, labnix at 02:30:14 and 13:26:17.
Mitigation	100310 kept at level 5; `100310` included in `FIM_FILE_RULES` of `logic_fim.py`; description with `[base: rule 554]` for analyst reference.
Origin	LAB FINDING [7] of the rules file (parent and child suppression)

10.2 Operation and Integration (AO)

AO-1 Field name="file", not syscheck.path RESOLVED Rule configuration	
Observation	In custom FIM rules, the correct field for path matching is `<field name="file">`. The `syscheck.path` field is not recognized by analysisd.
Impact	Silent failure: analysisd reports no error, the rule simply does not match.

AO-2 Syscheck saturation by noise sources RESOLVED Endpoint configuration	
Observation	Chrome (`.crdownload`), Trend Micro DirectPass, Sublime Text and MobaXterm write at high frequency to the monitored directories. Without a filter, the event volume exceeds what is useful to detect.
Mitigation	`<ignore type="sregex">` in the agent `ossec.conf`, covering the specific paths of each noise source.

AO-3 auditpol required for whodata on a Windows workgroup RESOLVED Endpoint configuration	
Observation	On LABDESK, in a workgroup environment with no Group Policy, object access auditing set to "Success" (shown as "Êxito" on Portuguese systems) had to be enabled manually. Without it, the SACL configured by Wazuh is silently invalidated; Wazuh emits error 6021 and whodata collects no user or process data.

Mitigation	Enable the two subcategories:
	`auditpol /set /subcategory:"File System" /success:enable`
	`auditpol /set /subcategory:"Handle Manipulation" /success:enable`

Note	Wazuh auto-configuration is sufficient under normal conditions (section 3.2); this finding records the behavior observed on LABDESK. Use GUIDs if the Windows language is not English.

AO-4 nginx client_max_body_size for LLM analysis payloads

RESOLVED Integration (nginx and Flask)

Observation	The POST to `/api/fim/analyze` (focal context, `file_events` and `process_events`) exceeds the nginx default limit of 1 MB, which returns HTTP 413 as an HTML page before passing the request to Flask.
Impact	The frontend fails with `Unexpected token '<'` when running `JSON.parse` on the response.
Mitigation	`client_max_body_size 10m;` in the `server {}` block of the nginx file `wptv`. Validated on October 8, 2026.

AO-5 AbuseIPDB is only queried via the auditd connect syscall

RESOLVED Integration (enrichment)

Observation	The pipeline extracts IPs from `data.audit.addr` in process events. IPs present in the content of monitored files are not extracted.
Mitigation	auditd rules for the `connect` syscall on labnix (`uid>=1000` and `uid=0`), loaded with `augenrules --load` and confirmed with `auditctl -l` (section 6.4).

AO-6 VirusTotal does not detect custom payloads

EXPECTED Integration (enrichment)

Observation	VirusTotal returned 0 detections for the `beacon_test.sh` created in the lab (SHA256 starting with `5ca772c3`). Custom or unseen scripts have no prior signature.
Interpretation	VirusTotal is a secondary signal. For new payloads, AbuseIPDB via the `connect` syscall is the primary signal.

AO-7 SACL not restored after a manager restart

MITIGATED Windows platform

Scenario	Manager restarted at 01:47:38 (October 10, 2026). The LABDESK agent loses the connection; with `auto_restart=yes` and `time-reconnect=60`, the WazuhSvc process restarts and reconnects. After reconnection, whodata on `D:\File_Server_Test_Folder` generated no syscheck events.
Timeline	01:47:38 Manager restart 02:10:35 File created in `D:\File_Server_Test_Folder`, with no syscheck event in `archives.log` 02:13:04 EventID 4719 (File System and Handle Manipulation, "Success added", account LABDESK$ S-1-5-18): SACL reapplied only after a manual `Restart-Service WazuhSvc` 02:14:29 File on Desktop (realtime) captured normally
Root cause	`windows_audit_interval=60` checks the audit policy and the SACLs while the whodata engine is running and, on divergence, switches the directories to realtime (official documentation); it does not reapply the SACL. In the lab, the SACL was reapplied only by the agent restart (EventID 4719 at 02:13:04). The agent restart on reconnection was inferred in the lab and is not documented by Wazuh. Total gap of about 25 minutes without whodata coverage.
Mitigation	After a manager restart, restart the agent on the affected Windows endpoint. For automation, consider a monitoring script that detects the absence of EventID 4719 after reconnection and restarts the service.

Note	The Desktop (realtime) was not affected: `ReadDirectoryChangesW` does not depend on the SACL and is not interrupted by reconnection.

AO-8 Path with /* expands the glob into individual files

RESOLVED Endpoint configuration

Scenario	Paths `/home/kr/Downloads/*` and `/home/kr/Desktop/*` temporarily configured on labnix.

Evidence with /*	ossec.log at 01:35:57:
	`(6016): Directory set for real time monitoring: '/home/kr/Downloads/file.zip'`
	`(6016): Directory set for real time monitoring: '/home/kr/Downloads/pewpewpew.txt'`
	`syscheckd` expanded the glob and registered watches on existing files; new files had no watch and inotify did not capture them.

Evidence without /*	ossec.log at 02:28:18:
	`(6003): Monitoring path: '/home/kr/Downloads', with options '... | realtime'`
	`(6016): Directory set for real time monitoring: '/home/kr/Downloads'`
	Watch registered on the directory; new files captured correctly.

Mitigation	Remove `/*` from the directory paths in `<directories>`.

11. Reference Configurations

11.1 Wazuh 4.14.8: FIM Updates

Version 4.14.8, released on September 23, 2026, brought improvements and fixes to the FIM module, especially for the whodata provider via eBPF on Linux. The changes affect the quality of integrity alerts, the accuracy of the audit fields and the stability of the pipeline described in this handbook.

Item	Type	Change	Impact
#32467	Addition	`audit_uid`, `audit_name`, `audit_gid` and `audit_group_name` fields in whodata FIM alerts from the eBPF provider	User and group identity without resolving the UID manually; fields verifiable in `syscheck.audit.*` in Discover
#39358	Fix	Swap between `user_id` and `group_id` in whodata FIM alerts from the eBPF provider	Rules that filter by a specific UID now receive the correct value
#39335	Fix	Attribution of all modifications to the root user when the kernel did not report `loginuid`	eBPF handles the absence of `loginuid`, avoiding wrong attribution to root
#38145	Fix	Out-of-bounds heap buffer write in `wazuh-analysisd` when generating FIM alerts; the `full_log` buffer is now resized before writing	Eliminates the risk of memory corruption with large payloads; relevant for high-volume FIM

Table 25 - 4.14.8 changes relevant to FIM (official release notes)

11.2 Hardened ossec.conf: Linux (labnix)

Excerpt of the `syscheck` block validated on labnix (Ubuntu 24.04.5, kernel 6.17.0, agent 4.14.8). The server address is a placeholder.

```
<client>
  <server>
    <address>YOUR-WAZUH-SERVER-IP</address>
    <port>1514</port>
    <protocol>tcp</protocol>
  </server>
  <notify_time>20</notify_time>
  <time-reconnect>60</time-reconnect>
  <auto_restart>yes</auto_restart>
</client>

<syscheck>
  <disabled>no</disabled>
  <frequency>60</frequency>
  <scan_on_start>yes</scan_on_start>

  <directories check_all="yes" whodata="yes">/etc</directories>
  <directories check_all="yes" whodata="yes">/usr/bin,/usr/sbin</directories>
  <directories check_all="yes" whodata="yes">/bin,/sbin,/boot</directories>

  <directories check_all="yes" realtime="yes">/home/kr/Downloads</directories>
  <directories check_all="yes" realtime="yes">/home/kr/Desktop</directories>
  <directories check_all="yes" realtime="yes">/tmp</directories>
  <directories check_all="yes" realtime="yes">/var/tmp</directories>

  <ignore type="sregex">.log$|.swp$</ignore>
  <nodiff>/etc/ssl/private.key</nodiff>
  <skip_nfs>yes</skip_nfs>
  <skip_dev>yes</skip_dev>
  <skip_proc>yes</skip_proc>
  <skip_sys>yes</skip_sys>
  <process_priority>10</process_priority>
  <max_eps>50</max_eps>
```

```
  <synchronization>
    <enabled>yes</enabled>
    <interval>5m</interval>
    <max_eps>10</max_eps>
  </synchronization>
</syscheck>

<localfile>
  <log_format>audit</log_format>
  <location>/var/log/audit/audit.log</location>
</localfile>
```

Notes: Directory paths are declared without `/*` (Finding AO-8). The `frequency` of 60 s is a lab value, used to speed up validation of the scenarios; in production, the default is 43200 s. The synchronization values shown are Wazuh defaults.

The auditd rule for syscheck covers `connect` in `/etc/audit/rules.d/net_connect.rules` (section 6.4).

11.3 Hardened ossec.conf: Windows (LABDESK)

Excerpt of the `syscheck` block validated on LABDESK (Windows 11 Pro, agent 4.14.8), keeping the default Wazuh paths and adding the malware staging and persistence paths. The synchronization values shown are Wazuh defaults.

```
<syscheck>
  <disabled>no</disabled>
  <directories realtime="yes">%PROGRAMDATA%\Microsoft\Windows\Start
Menu\Programs\Startup</directories>

  <directories check_all="yes" realtime="yes">C:\Users\kr\Downloads</directories>
  <directories check_all="yes" realtime="yes"
report_changes="yes">C:\Users\kr\AppData\Local\Temp</directories>
  <directories check_all="yes" realtime="yes"
report_changes="no">C:\Users\kr\AppData\Roaming</directories>
  <directories check_all="yes" realtime="yes"
report_changes="yes">C:\Users\kr\Desktop</directories>
  <directories check_all="yes" realtime="yes" report_changes="yes">C:\Windows\Temp</directories>

  <directories check_all="yes" whodata="yes"
report_changes="no">C:\Windows\System32</directories>
  <directories check_all="yes" whodata="yes"
report_changes="no">C:\Windows\SysWOW64</directories>
  <directories check_all="yes" whodata="yes"
report_changes="no">C:\Windows\System32\drivers</directories>
  <directories check_all="yes" whodata="yes"
report_changes="no">C:\Windows\System32\wbem</directories>
  <directories check_all="yes" whodata="yes"
report_changes="no">C:\Windows\System32\WindowsPowerShell</directories>

  <directories check_all="yes" whodata="yes"
report_changes="yes">D:\File_Server_Test_Folder</directories>

  <windows_registry
arch="both">HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run</windows_registry>
  <windows_registry
arch="both">HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run</windows_registry>

  <ignore type="sregex">.etl$|.tmp$|.~$</ignore>
  <windows_audit_interval>60</windows_audit_interval>
  <process_priority>10</process_priority>
  <max_eps>50</max_eps>
  <synchronization>
    <enabled>yes</enabled>
    <interval>5m</interval>
    <max_eps>10</max_eps>
```

```
  </synchronization>
</syscheck>
```

11.4 Server State (WZ-SRV)

File	Path	State
`fim_anomaly_rules.xml`	`/var/ossec/etc/rules/`	6 rules (100310 and 100300 to 100304); 100310 at level 5; permissions 660 wazuh:wazuh
`fim_sysmon_rules.xml`	`/var/ossec/etc/rules/`	5 rules (100521, 100540, 100541, 100554 and 100555); permissions 660 wazuh:wazuh
`ossec.conf` (manager)	`/var/ossec/etc/`	`logall_json` yes; filebeat archives enabled; syscheck `frequency` 60
`logic_fim.py`	`/usr/share/wazuh-dashboard/plugins/process_tree_api/`	`FIM_FILE_RULES` with 100310; `_enrich_intel()` with VirusTotal, OTX and AbuseIPDB; `max_tokens` 4096; handler for `JSONDecodeError`
`server.py`	`process_tree_api/`	Routes `/api/fim/*`
`fim.html`	`process_tree_api/public/`	Panel with descending sort, hostname search, free-text search and resizable divider
`wptv` (nginx)	`/etc/nginx/sites-available/`	`client_max_body_size 10m`

Table 26 - State of the lab files on the server

11.5 Conclusion

FIM is not an isolated product. It is a native capability of the Wazuh agent, integrated into an ecosystem that collects, decodes, correlates, indexes and visualizes security data. A single agent covers files and the Windows Registry, identifies who made a change through whodata, maps events to compliance requirements and delivers everything into the SIEM and XDR flow, all on an open and free platform.

The value of FIM, however, depends on design and validation, not only on activation. The 15 findings in chapter 10 show that coverage can disappear without any error in the logs (AO-7, AO-8), and that the cause is often one layer below the agent: an operating system audit policy or a forgotten GPO. They are lab observations on the 4.14.x series, each one documented with evidence and mitigation.

A well-configured FIM starts with paths chosen by criteria (malware staging areas, persistence locations, system binaries), whodata where attribution matters, noise controlled with ignore, correlation rules tuned against a baseline, and continuous verification of coverage (agent lifecycle rules and rule 80705). On top of that, correlation, Sysmon as a complementary layer and AI-assisted investigation shorten the time from alert to understanding, with the analyst always making the decision. In the lab, the measured alert-to-verdict latency was 16 seconds.

For organizations of any size the path is the same, with different emphasis. In smaller environments, a few critical directories combined with the Wazuh defaults and compliance mapping already deliver visibility and audit evidence. In medium and large environments, agent groups and centralized configuration allow profiles per role, such as file servers, build servers and backup servers, each with its own volume and threshold tuning.

The values in this Handbook come from a lab with 3.9 GB of RAM and are not a sizing guide, so each environment needs its own baseline.

FIM is not install and forget. It is a control that needs design, validation and maintenance. When it gets them, it becomes one of the most accessible ways to turn file integrity into detection, response and compliance signal, within reach of any team that adopts Wazuh.

I hope this advanced handbook will make a meaningful contribution to the entire Wazuh community, supporting users, analysts, and security professionals at every level of expertise. My goal is to share practical knowledge, validated research, real-world lab findings, and technical insights that help readers deepen their understanding of File Integrity Monitoring (FIM), strengthen their security monitoring capabilities, improve detection and incident investigation, and make more effective use of Wazuh in their own environments.

Beyond serving as a technical reference, I hope this work encourages curiosity, critical thinking, knowledge sharing, and continuous learning. Whether you are taking your first steps with Wazuh or have years of experience in cybersecurity, I hope this handbook provides valuable insights, practical guidance, and ideas that inspire further research and contribute to a stronger, more knowledgeable, and collaborative security community.

12. Official References

The documentation and reference links below were reopened and confirmed on October 10 and October 12, 2026. The last row is this project's repository.

Reference	Address	Status
Release notes 4.14.8	https://documentation.wazuh.com/current/release-notes/release-4-14-8.html	**Verified**
Custom FIM rules	https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/creating-custom-fim-rules.html	**Verified**
Malware persistence detection	https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/use-cases/detecting-malware-persistence-technique.html	**Verified**
FIM: overview	https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/index.html	**Verified**
FIM: how it works	https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/how-it-works.html	**Verified**
FIM: basic settings	https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/basic-settings.html	**Verified**
FIM: advanced settings	https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/advanced-settings.html	**Verified**
Windows Registry monitoring	https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/windows-registry-monitoring.html	**Verified**

Reference	Address	Status
ossec.conf reference: syscheck	https://documentation.wazuh.com/current/user-manual/reference/ossec-conf/syscheck.html	**Verified**
System calls monitoring	https://documentation.wazuh.com/current/user-manual/capabilities/system-calls-monitoring/how-it-works.html	**Verified**
Audit configuration	https://documentation.wazuh.com/current/user-manual/capabilities/system-calls-monitoring/audit-configuration.html	**Verified**
Centralized configuration	https://documentation.wazuh.com/current/user-manual/reference/centralized-configuration.html	**Verified**
PCI DSS compliance and FIM	https://documentation.wazuh.com/current/compliance/pci-dss/file-integrity-monitoring.html	**Verified**
HIPAA compliance and FIM	https://documentation.wazuh.com/current/compliance/hipaa/file-integrity-monitoring.html	**Verified**
NIST compliance and FIM	https://documentation.wazuh.com/current/compliance/nist/file-integrity-monitoring.html	**Verified**
Active Response (official documentation)	https://documentation.wazuh.com/current/user-manual/capabilities/	**Verified**
MITRE ATT&CK T1485	https://attack.mitre.org/techniques/T1485/	**Verified**
MITRE ATT&CK T1486	https://attack.mitre.org/techniques/T1486/	**Verified**
WAZUH-Process-Tree-Viewer	https://github.com/mym0us3r/WAZUH-Process-Tree-Viewer	**Verified**
Wazuh FIM - Advanced Defender's Handbook - This project	https://github.com/mym0us3r/wazuh-fim-adh	**This project**

Table 27 - Official references and verification status

Kislley Rodrigues - Wazuh Ambassador Program
[October 2026 - Wazuh FIM: The Advanced Defender's Handbook]
LinkedIn: https://www.linkedin.com/in/rkbr/
GitHub Repository: https://github.com/mym0us3r/wazuh-fim-adh

More information about Wazuh:

wazuh.

Wazuh - Open Source XDR and SIEM



Wazuh | Ambassadors Program